Министерство цифрового развития, связи и массовых коммуникаций РФ
Уральский технический институт связи и информатики (филиал)
ФГБОУ ВО «Сибирский государственный университет
телекоммуникаций и информатики» в г. Екатеринбурге
(УрТИСИ СибГУТИ)

Отчёт
по учебной практике
по МДК 02.02 «Инструментальные средства разработки
программного обеспечения»
на тему «Разработка приложения по варианту ДЭ
и реализация дипломного проекта»

студента IV курса 284 группы

Фамилия Гусев

Имя, отчество Михаил Алексеевич

Факультет Инфокоммуникаций, информатики и управления

Специальность 09.02.07 «Информационные системы
и программирование»

Екатеринбург, 2026

Отзыв руководителя

Содержание

Введение 4
1 Демонстрационный экзамен 5
1.1 Анализ предметной области 5
1.2 Выбор стека технологий и инструментов разработки 5
1.3 Работа с базой данных 6
1.4 Практическая реализация решения 7
1.5 Интерфейс программы 9
1.6 Качественные характеристики кода 13
1.7 Тестирование программы 15
1.8 Руководство системному программисту 16
2 Дипломный проект 20
2.1 Введение по дипломному проекту 20
2.2 Анализ предметной области 21
2.3 Требования к программному продукту 22
2.4 Обоснование выбора инструментальных средств разработки 23
2.5 Проектирование и разработка базы данных 24
2.6 Проектирование и реализация архитектуры Web API 27
2.7 Инструкция по работе с веб приложением 30
Заключение 40
Библиография 41
Приложение А 42

Введение

Данная учебная практика направлена на подготовку к демонстрационному экзамену и описанию практической части дипломного проекта.

Цель учебной практики – проработать вариант демонстрационного экзамена и составить пояснительную записку к практической части дипломного проекта.

Для достижения поставленной цели необходимо решить следующие задачи:

– проанализировать техническое задание и предметную область мебельной компании;

– спроектировать базу данных в 3 нормальной форме;

– разработать приложение на платформе ASP.NET Core MVC;

– реализовать функционал просмотра, добавления, редактирования продукции, просмотра цехов и расчёта сырья;

– протестировать программу;

– подготовить документацию к программному продукту.

1 Демонстрационный экзамен

1.1 Анализ предметной области

Разрабатываемый модуль предназначен для автоматизации учёта продукции мебельной компании «Комфорт». Согласно техническому заданию, программа должна обеспечивать следующий функционал:

– просмотр списка продукции;

– добавление и редактирование данных о продукции;

– просмотр списка цехов для производства конкретного продукта;

– расчёт количества сырья для производства заданного объёма продукции с учётом коэффициента типа продукции и процента потерь материала.

Нефункциональные требования включают:

– приведение базы данных к третьей нормальной форме;

– единый стиль интерфейса в соответствии с руководством (шрифт Candara, цветовая гамма #FFFFFF, #D2DFFF, #355CBD);

– обработка исключительных ситуаций с понятными сообщениями пользователю.

Входные данные:

– данные о продукции (артикул, тип, наименование, минимальная стоимость, основной материал);

– данные о цехах (название, тип, количество человек);

– параметры для расчёта сырья (тип продукции, тип материала, количество, два вещественных параметра).

Выходные данные:

– список продукции с общим временем изготовления;

– детальный список цехов для выбранного продукта;

– результат расчёта сырья (целое число) либо сообщение об ошибке.

1.2 Выбор стека технологий и инструментов разработки

Для реализации программного продукта выбран стек технологий на платформе .NET. В качестве языка программирования использован C#, среда разработки – Visual Studio 2022. Эти средства обеспечивают удобство разработки, отладки и богатую библиотечную поддержку.

Для создания веб-интерфейса применён фреймворк ASP.NET Core MVC версии 8.0. Он позволяет чётко разделить логику приложения на модели, представления и контроллеры, упрощает обработку форм и валидацию данных.

Система управления базами данных – Microsoft SQL Server (LocalDB). Выбор обусловлен простотой развёртывания и полной интеграцией с Entity

Framework Core. Для администрирования использовалась среда SSMS (SQL Server Management Studio).

Доступ к данным организован через Entity Framework Core (подход Database First). Это позволило быстро создать модели по существующей базе данных и выполнять запросы с помощью LINQ.

На стороне клиента использованы Bootstrap 5 для стилизации, кастомные CSS-классы для соблюдения фирменного стиля и JavaScript для асинхронной загрузки данных при редактировании.

Система контроля версий – Git, репозиторий размещён на GitHub. Это обеспечило ежечасное отслеживание изменений и резервное копирование кода.

1.3 Работа с базой данных

На основе анализа предметной области и предоставленных файлов импорта (Product_type_import.xlsx, Material_type_import.xlsx, Workshops_import.xlsx, Products_import.xlsx, Product_workshops_import.xlsx) были выделены следующие сущности:

– ProductTypes (типы продукции): ProductTypeId (PK), Name, Coefficient;

– MaterialTypes (типы материалов): MaterialTypeId (PK), Name, LossPercent;

– Workshops (цеха): WorkshopId (PK), Name, Type, EmployeeCount;

– Products (продукция): ProductId (PK), ProductTypeId (FK), Name, Article, MinPriceForPartner, MainMaterialId (FK);

– ProductWorkshops (связь продукции с цехами): ProductWorkshopId (PK), ProductId (FK), WorkshopId (FK), TimeHours.

Связи между сущностями:

– один тип продукции может быть у многих продуктов (ProductTypes → Products 1:M);

– один тип материала может быть основным для многих продуктов (MaterialTypes → Products 1:M);

– один продукт может проходить через много цехов, один цех может использоваться для многих продуктов (Products ↔ Workshops M:N через ProductWorkshops);

– при удалении продукта каскадно удаляются связи в ProductWorkshops.

База данных приведена к третьей нормальной форме: отсутствуют транзитивные зависимости, все неключевые атрибуты зависят только от первичного ключа.

ER-диаграмма разработанной базы данных представлена на рисунке 1.1.

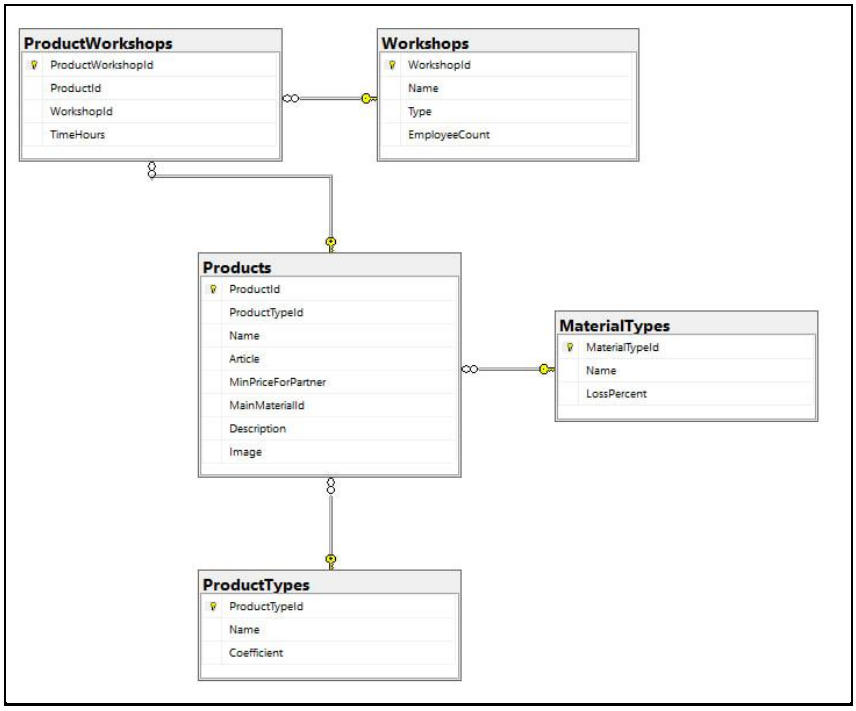


Рисунок 1.1 – ERD-диаграмма

1.4 Практическая реализация решения

Программное обеспечение реализовано в виде веб-приложения на основе шаблона MVC, что позволяет разделять ответственности между компонентами системы.

– Model: сущности базы данных реализованы классами Product, ProductType, MaterialType, Workshop, ProductWorkshop. Для привязки сущностей к таблицам применены атрибуты [Table("...")] и [Key]. Между сущностями установлены связи: продукт относится к одному типу продукции и одному основному материалу, продукт может проходить через множество цехов (связь многие-ко-многим через промежуточную таблицу ProductWorkshop, которая также хранит время изготовления);

– Data: для работы с БД используется ApplicationDbContext, он содержит наборы сущностей (DbSet<Product>, DbSet<ProductType> и т.д.) и конфигурирует связи через OnModelCreating, включая установку каскадного удаления для ProductWorkshop (при удалении продукта связанные записи удаляются автоматически);

– View: пользовательский интерфейс реализован Razor-представлениями. Главная страница (Index) отображает таблицу продукции с кнопками действий, данные передаются через модель IEnumerable<ProductIndexViewModel>. Для добавления и редактирования используется одно модальное окно, которое заполняется либо пустой формой, либо данными, полученными асинхронно с

сервера. Страница Workshops отображает список цехов для выбранного продукта, а страница Calculate предоставляет интерфейс для расчёта сырья;

– Controller: основная логика работы с продукцией реализована в ProductsController. Он содержит методы для отображения списка (Index), получения данных продукта для редактирования (GetProductForEdit), сохранения (SaveProduct), отображения цехов (Workshops), а также для расчёта сырья (Calculate – GET и POST). Дополнительные контроллеры не требуются, так как весь функционал сосредоточен вокруг продукции;

– сервис расчёта сырья: для вычисления количества необходимого сырья создан отдельный сервис RawMaterialCalculator, реализующий интерфейс IRawMaterialCalculator. Он принимает идентификаторы типа продукции и материала, количество и два вещественных параметра, обращается к справочным таблицам через ApplicationDbContext и возвращает целое число (или -1 при ошибке). Сервис зарегистрирован в контейнере внедрения зависимостей и используется в ProductsController.

Работа с продукцией реализована с помощью контроллера ProductsController, содержащего набор операций:

– список продукции (Index): формируется запрос к _context.Products с подгрузкой связанных типов и цехов, вычисляется общее время изготовления суммированием TimeHours и округлением вверх, результат проецируется в модель ProductIndexViewModel. Для выпадающих списков в модальном окне дополнительно загружаются справочники ProductTypes и MaterialTypes и помещаются в ViewBag;

– получение данных для редактирования (GetProductForEdit): по запросу из JavaScript возвращает JSON с данными продукта, которые используются для заполнения модальной формы;

– сохранение продукции (SaveProduct): принимает данные из формы, выполняет проверку валидации (ModelState.IsValid), проверяет существование выбранных типов и материалов. Если ProductId равен 0 – создаётся новый продукт, иначе обновляется существующий. При успехе сохраняет изменения в БД и передаёт сообщение через TempData["SuccessMessage"], при ошибке – TempData["ErrorMessage"] с описанием;

– просмотр цехов (Workshops): по идентификатору продукта загружает его с включением связанных записей ProductWorkshops и Workshops, формирует модель ProductWorkshopsViewModel и передаёт в представление. Если продукт не найден, пользователь перенаправляется на главную с сообщением об ошибке;

– расчёт сырья (Calculate – GET): отображает форму, заполняя выпадающие списки типами продукции и материала из БД;

– расчёт сырья (Calculate – POST): принимает заполненную модель, проверяет валидацию, вызывает метод сервиса RawMaterialCalculator. Результат помещается в Model.Result и возвращается в представление для отображения. При ошибке валидации или возврате -1 выводится соответствующее сообщение.

Для повышения удобства использования реализованы обработка ошибок и информационные сообщения:

– проверки NotFound, BadRequest и другие для корректной обработки отсутствующих данных;

– обработка исключений при сохранении в БД (try-catch с DbUpdateException и общим Exception) с выводом понятного сообщения через TempData["ErrorMessage"];

– вывод информационных сообщений пользователю через TempData["SuccessMessage"] и TempData["ErrorMessage"] с отображением в виде цветных Bootstrap-уведомлений на главной странице;

– клиентская валидация с подсветкой ошибочных полей и отменой отправки формы при неверных данных.

JavaScript обеспечивает асинхронную загрузку данных для редактирования (fetch) и управление модальным окном Bootstrap. Это позволяет реализовать редактирование без перезагрузки страницы и с минимальным количеством представлений.

1.5 Интерфейс программы

В процессе разработки программы было реализовано следующее:

– просмотр списка продукции в виде таблицы с колонками: артикул, тип продукции, наименование, минимальная стоимость для партнёра, общее время изготовления;

– добавление новой продукции через модальное окно с валидацией обязательных полей (артикул, тип, наименование, стоимость, основной материал);

– редактирование существующей продукции с предварительной загрузкой данных через AJAX и открытием того же модального окна;

– просмотр списка цехов для выбранного продукта с указанием названия цеха, количества человек и времени изготовления в каждом цехе;

– расчёт количества сырья для производства заданного объёма продукции с учётом коэффициента типа продукции и процента потерь материала (реализован отдельный сервис, возвращающий целое число или -1 при ошибке);

– единый стиль интерфейса в соответствии с руководством (шрифт Candara, основной фон #FFFFFF, дополнительный фон #D2DFFF, акцентный цвет #355CBD);

– обработка исключительных ситуаций с выводом понятных сообщений пользователю через всплывающие уведомления (Bootstrap alerts);

– навигация между страницами с помощью кнопок «Назад» и ссылок в шапке сайта;

Интерфейс программы представлен на рисунках 1.2-1.6.

В верхней части расположен заголовок «Продукция компании» и логотип организации. Ниже находятся две кнопки: «Добавить продукцию» для создания новой записи и «Расчёт сырья» для перехода к калькулятору. Основное пространство занимают карточки товаров, каждая из которых содержит информацию о продукте: артикул, стоимость для партнёра, основной материал и общее время изготовления. В правом нижнем углу каждой карточки расположены кнопки действий: редактирование (иконка карандаша), просмотр цехов (иконка завода) и удаление (иконка корзины).

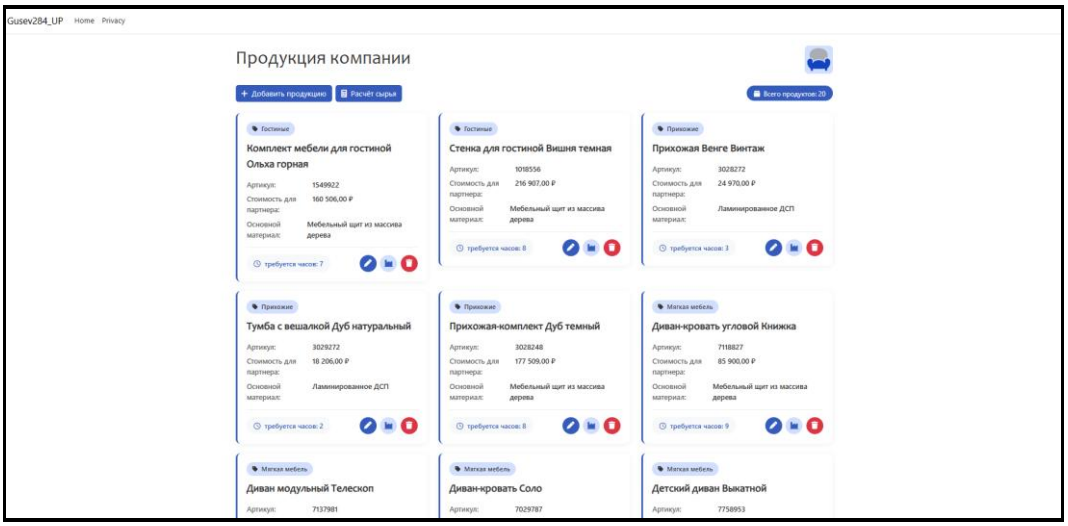


Рисунок 1.2 – Основная вкладка

На рисунке показано модальное окно, открывающееся при нажатии кнопки «Добавить продукцию». Форма содержит обязательные для заполнения поля: артикул (текстовое поле), тип продукции (выпадающий список), наименование (текстовое поле), минимальная стоимость для партнёра (числовое поле) и основной материал (выпадающий список). Поля, отмеченные звёздочкой (*), являются обязательными. В нижней части окна расположены кнопки «Отмена» для закрытия окна без сохранения и «Сохранить» для добавления новой записи в базу данных. Все поля валидируются как на стороне клиента, так и на сервере.

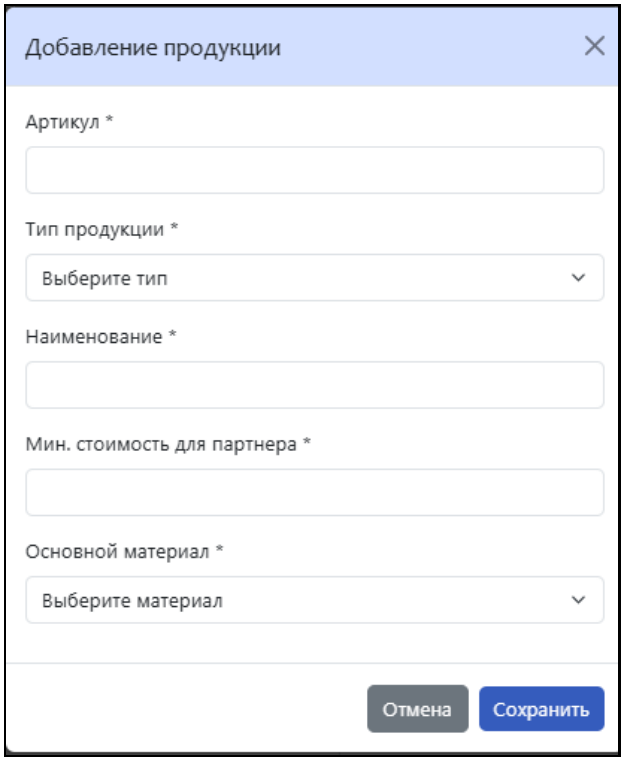


Рисунок 1.3 – Окно для добавления продукции

На странице расположена форма для ввода параметров расчёта: тип продукции, тип материала, количество продукции и два вещественных параметра. После заполнения полей и нажатия кнопки «Рассчитать» система выполняет вычисления с учётом коэффициента типа продукции и процента потерь материала. Результат отображается справа в виде целого числа — необходимого количества сырья. При ошибке ввода или несуществующих типах выводится соответствующее сообщение. В верхней части страницы предусмотрена кнопка возврата к списку продукции.

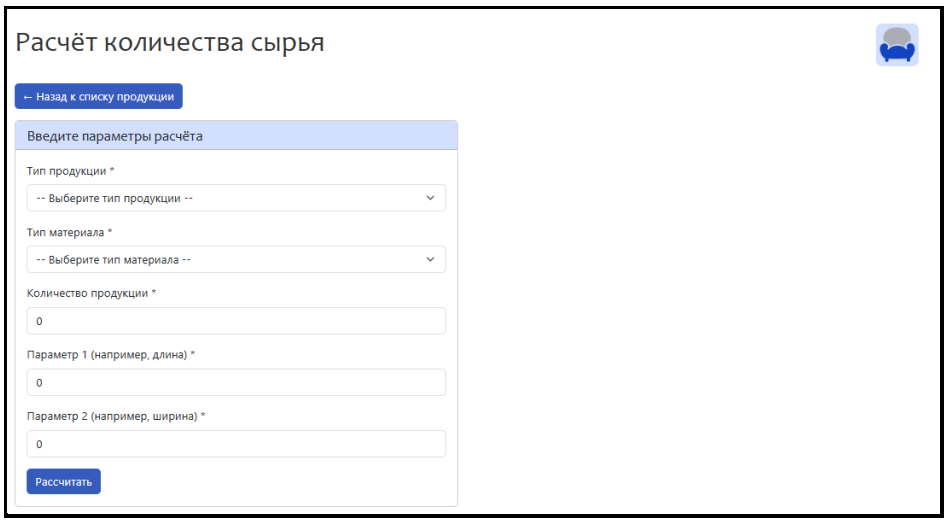


Рисунок 1.4 – Вкладка для расчета количества сырья

На рисунке представлено модальное окно редактирования, которое открывается при нажатии на иконку карандаша в карточке товара. В отличие от окна добавления, все поля предварительно заполнены данными выбранного продукта. В данном примере редактируется продукт «Комплект мебели для гостиной Ольха горная» с артикулом 1549921. Пользователь может изменить любое поле и сохранить изменения, нажав кнопку «Сохранить», либо отменить редактирование кнопкой «Отмена». Загрузка данных для редактирования выполняется асинхронно через AJAX-запрос.

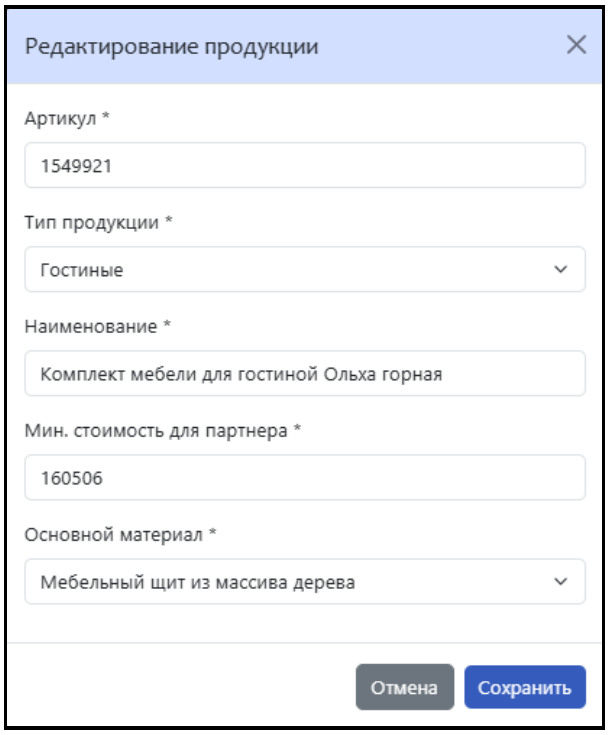


Рисунок 1.5 – Окно для редактирования ячейки продукции

На рисунке изображено стандартное диалоговое окно браузера с запросом подтверждения удаления продукта. Система предупреждает пользователя: «Удалить продукт? Все связанные данные будут также удалены». Это предупреждение появляется при нажатии на иконку корзины в карточке товара и защищает от случайного удаления записей. При нажатии «OK» продукт безвозвратно удаляется из базы данных вместе со связанными записями в таблице ProductWorkshops (благодаря каскадному удалению). При нажатии «Отмена» удаление отменяется, и пользователь возвращается к списку продукции.

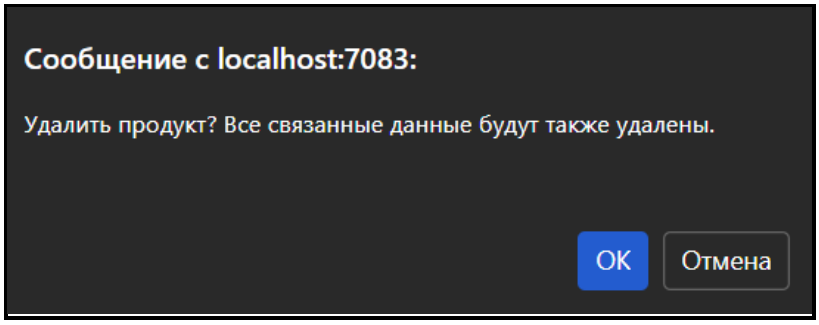
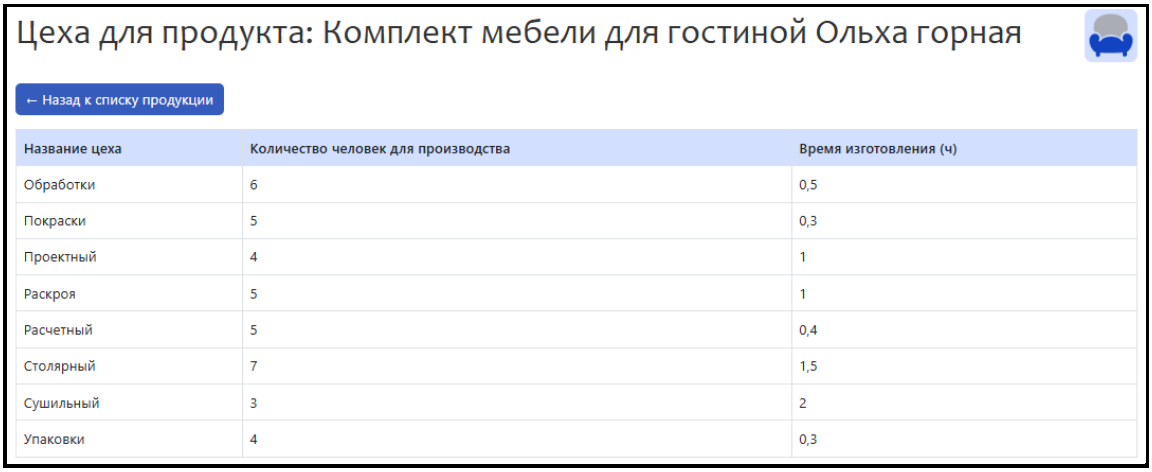


Рисунок 1.6 – Предупреждение при удалении

На рисунке представлена страница «Цеха для продукта: Комплект мебели для гостиной Ольха горная». В верхней части расположена кнопка «← Назад к списку продукции» для возврата на главную страницу. Основное содержание страницы — таблица с перечнем цехов, задействованных в производстве данного продукта. Таблица содержит три колонки: название цеха, количество человек для производства и время изготовления в часах. Для продукта «Комплект мебели для гостиной Ольха горная» отображается 8 цехов с различными временными затратами. Данные для таблицы формируются на основе связей в базе данных через промежуточную таблицу ProductWorkshops.



Название цеха	Количество человек для производства	Время изготовления (ч)
Обработки	6	0,5
Покраски	5	0,3
Проектный	4	1
Раскроя	5	1
Расчетный	5	0,4
Столярный	7	1,5
Сушильный	3	2
Упаковки	4	0,3

Рисунок 1.7 – Вкладка цеха

1.6 Качественные характеристики кода

После тестирования проводилась оценка качественных характеристик разработанного кода:

1) полнота обработки ошибочных данных:

– присутствуют: проверки на null и отсутствие сущностей (NotFound, FirstOrDefault с проверкой), проверка существования связанных записей

(ProductTypeExists, MaterialTypeExists), обработка исключений при сохранении в БД (try-catch с DbUpdateException и общим Exception), валидация модели через ModelState.IsValid, проверка положительности стоимости (Range атрибут), клиентская валидация с подсветкой ошибочных полей;

– отсутствуют: нет централизованного логирования ошибок, нет обработки специфических исключений БД (например, нарушение уникальности), ограничения по длине строк установлены только через атрибуты StringLength;

– итог: обработка ошибок реализована на достаточном для демо-экзамена уровне, но может быть расширена в будущем;

2) наличие тестов для проверки допустимых значений входных данных:

– тесты отсутствуют, так как разработка велась без использования модульного тестирования в рамках учебной практики;

3) наличие средств контроля корректности входных данных:

– присутствуют: использование атрибутов валидации ([Required], [Range], [StringLength]) в моделях представлений, проверка ModelState.IsValid в контроллере, дополнительные ручные проверки (существование выбранных типов продукции и материалов), клиентская валидация на JavaScript с подсветкой полей;

– отсутствуют: нет кастомных атрибутов валидации для сложных бизнес-правил;

– итог: контроль входных данных реализован на базовом уровне и покрывает основные сценарии;

4) наличие средств восстановления при сбоях оборудования:

– отсутствуют: в приложении не реализованы автоматическое резервное копирование, транзакции с откатом (используется стандартное поведение EF Core SaveChanges внутри одной транзакции), механизмы повторных попыток подключения к БД;

– итог: средства восстановления не предусмотрены, что допустимо для учебного проекта;

5) наличие комментариев:

– присутствуют: краткие комментарии над методами контроллера с указанием модуля и назначения (например, "МОДУЛЬ 2: Главная страница со списком"), комментарии в сложных местах JavaScript (функции clearModal, openEditModal, валидация), пояснения в сервисе расчёта сырья;

– отсутствуют: избыточные комментарии к очевидному коду не добавлялись;

– итог: комментарии присутствуют в достаточной мере и описывают назначение основных компонентов;

6) наличие проверки корректности передаваемых данных:

– присутствуют: проверка существования продукта по ID перед редактированием и удалением (FindAsync, FirstOrDefault), проверка наличия связанных записей (ProductWorkshops) перед отображением, проверка параметров маршрута (productId) на валидность;

– итог: проверка корректности передаваемых данных реализована в полном объёме;

7) наличие описаний основных функций:

– присутствуют: все основные функции имеют понятные имена в соответствии с соглашением CamelCase (Index, GetProductForEdit, SaveProduct, Delete, Workshops, Calculate), сопровождаются краткими комментариями о назначении. Названия методов и переменных отражают их смысл (например, TotalTimeHours, MainMaterialName, ProductTypeId);

– итог: описание основных функций присутствует на уровне кода через имена и краткие комментарии.

1.7 Тестирование программы

Тестирование программного продукта проводилось методом функционального тестирования - проверки соответствия реализованного функционала требованиям технического задания. Для каждого тестового случая определялись: входные данные, ожидаемый результат и фактический результат.

Всего было выполнено 20 тестовых случаев. Результаты тестирования представлены в таблице 1.1.

Таблица 1.1 – Результаты функционального тестирования

№	Описание теста	Ожидаемый результат	Фактический результат
1	Открыть главную страницу /Products	Отображение списка продукции в виде карточек или таблицы	Успешно, список загружен
2	Проверить отображение логотипа и заголовка	Логотип и заголовок "Продукция компании" видны	Успешно, логотип и заголовок отображаются
3	Проверить счётчик количества продуктов	Отображение общего количества продуктов в правом верхнем углу	Успешно, количество совпадает с БД
4	Нажать кнопку "Добавить продукцию"	Открытие модального окна с пустой формой	Успешно, модальное окно открыто
5	Заполнить все поля корректными данными и сохранить	Появление сообщения "Продукт успешно добавлен", новый продукт в списке	Успешно, продукт добавлен
6	Оставить поле "Артикул" пустым и попытаться сохранить	Подсветка поля красным, форма не отправляется	Успешно, валидация сработала
7	Ввести отрицательную стоимость (-100)	Подсветка поля красным, форма не отправляется	Успешно, валидация не пропускает
8	Не выбрать тип продукции и попытаться сохранить	Подсветка выпадающего списка, форма не отправляется	Успешно, валидация сработала
9	Нажать кнопку редактирования (иконка карандаша) у продукта	Открытие модального окна с заполненными данными продукта	Успешно, данные подгружены
10	Изменить наименование продукта и сохранить	Сообщение "Продукт успешно обновлён", наименование изменилось в списке	Успешно, данные обновлены
11	Нажать кнопку удаления (иконка корзины)	Появление диалога подтверждения "Удалить продукт?"	Успешно, диалог отображается

12	Подтвердить удаление	Сообщение "Продукт успешно удалён", продукт исчезает из списка	Успешно, продукт удалён
13	Отменить удаление в диалоге подтверждения	Продукт остаётся в списке, сообщений нет	Успешно, удаление отменено
14	Нажать кнопку "Цеха" (иконка завода) у продукта	Переход на страницу с названием продукта и таблицей цехов	Успешно, страница загружена
15	Проверить отображение данных в таблице цехов	Отображение названия цеха, количества человек, времени изготовления	Успешно, данные соответствуют БД
16	Нажать кнопку "Назад к списку продукции"	Возврат на главную страницу со списком продуктов	Успешно, навигация работает
17	Перейти на страницу калькулятора (/Products/Calculate)	Отображение формы с выпадающими списками и полями ввода	Успешно, страница загружена
18	Выбрать тип продукции, тип материала, ввести количество 10, параметры 2.5 и 1.8, нажать "Рассчитать"	Отображение результата (целое число) в правой части экрана	Успешно, расчёт выполнен
19	Выбрать несуществующий ID продукции (если возможно через инспектор) или оставить поля пустыми	Сообщение об ошибке валидации или результат -1	Успешно, ошибка обработана
20	Проверить все ссылки и кнопки навигации между страницами	Все переходы работают корректно, страницы загружаются	Успешно, навигация стабильна

1.8 Руководство системному программисту

1.8.1 Общие сведения о программе

Программный модуль «ComfortFurniture» предназначен для учёта продукции мебельной компании. Система обеспечивает ведение справочника продукции, просмотр цехов, задействованных в производстве, и расчёт количества сырья для выпуска изделий.

Основные функции программы:

– просмотр, добавление, редактирование и удаление продукции;

– просмотр списка цехов для конкретного продукта с указанием времени изготовления и количества человек;

– расчёт необходимого количества сырья на основе типа продукции, типа материала, количества и двух вещественных параметров.

Технические средства:

– компьютер с операционной системой Windows 10/11 или Linux, поддерживающий .NET 8.0;

– СУБД Microsoft SQL Server (LocalDB, Express или полноценный сервер);

– веб-браузер для доступа к интерфейсу.

Программные средства:

– ASP.NET Core 8.0 MVC;

– Entity Framework Core (Microsoft.EntityFrameworkCore.SqlServer);

– Bootstrap 5 для интерфейса;

– система контроля версий Git.

1.8.2 Структура программы

Структура программы организована по шаблону MVC:

– точка входа Program.cs – регистрация сервисов (DbContext, сервис расчёта сырья), настройка маршрутизации и middleware;

– доступ к данным Data/ApplicationDbContext.cs – содержит DbSet<Product>, DbSet<ProductType>, DbSet<MaterialType>, DbSet<Workshop>, DbSet<ProductWorkshop>, настройка связей и индексов;

– модели Models/ – классы, соответствующие таблицам базы данных (Product, ProductType, MaterialType, Workshop, ProductWorkshop);

– модели представлений ViewModels/ – классы для передачи данных в представления (ProductIndexViewModel, ProductFormViewModel, ProductWorkshopsViewModel, RawMaterialCalculateViewModel);

– контроллеры Controllers/ProductsController.cs – содержит всю бизнес-логику: отображение списка, получение данных для редактирования, сохранение, удаление, просмотр цехов, расчёт сырья;

– сервисы Services/ – реализация интерфейса IRawMaterialCalculator в классе RawMaterialCalculator;

– представления Views/Products/ – Razor-шаблоны (Index.cshtml, Workshops.cshtml, Calculate.cshtml).

Связи между частями:

– контроллер получает ApplicationDbContext через внедрение зависимостей и выполняет запросы к БД;

– представления получают данные через строго типизированные модели или ViewBag для выпадающих списков;

– сервис расчёта сырья также использует DbContext для получения коэффициентов и процентов потерь.

1.8.3 Настройка программы

Настройка окружения:

1) установить .NET 8.0 SDK (скачать с официального сайта Microsoft);

2) установить SQL Server (можно использовать LocalDB, входящую в Visual Studio, или SQL Server Express);

3) убедиться, что строка подключения в appsettings.json указывает на корректный экземпляр SQL Server.

Настройка подключения к базе данных:

– в файле appsettings.json должна быть задана строка подключения DefaultConnection.

Инициализация базы данных:

1) выполнить скрипт создания таблиц (прилагается в файле Database.sql);

2) заполнить справочные таблицы данными из Excel-файлов (Product_type_import, Material_type_import, Workshops_import, Products_import, Product_workshops_import) с помощью приложенного скрипта-наполнения;

3) при необходимости создать резервную копию базы данных.

Настройка сервиса расчёта сырья:

– сервис RawMaterialCalculator автоматически регистрируется в Program.cs как Scoped;

– для корректной работы таблицы ProductTypes и MaterialTypes должны содержать актуальные коэффициенты и проценты потерь.

1.8.4 Проверка программы

Проверка работоспособности проводится через веб-интерфейс в браузере:

1) проверка отображения списка продукции:
   – открыть главную страницу /Products;
   – убедиться, что карточки товаров отображаются корректно;
2) проверка добавления продукции:
   – нажать кнопку «Добавить продукцию»;
   – заполнить форму и сохранить;
   – убедиться, что новый продукт появился в списке;
3) проверка редактирования:
   – нажать кнопку редактирования (иконка карандаша) у любого продукта;
   – изменить данные и сохранить;
   – убедиться, что данные обновились;
4) проверка удаления:
   – нажать кнопку удаления (иконка корзины) у любого продукта;
   – подтвердить удаление;
   – убедиться, что продукт исчез из списка;
5) проверка просмотра цехов:
   – нажать кнопку «Цеха» (иконка завода) у продукта;
   – убедиться, что отображается таблица с цехами;
6) проверка расчёта сырья:
   – перейти на страницу /Products/Calculate;
   – заполнить форму и нажать «Рассчитать»;
   – убедиться, что отображается результат (целое число).

1.8.5 Сообщения системному программисту

В процессе работы приложение может выдавать следующие сообщения в интерфейсе:

– «Продукт успешно добавлен» / «Продукт успешно обновлён» / «Продукт успешно удалён» – информируют об успешном выполнении операции;

– «Ошибка валидации: ...» – означает, что введённые данные не прошли проверку (не заполнены обязательные поля, отрицательная стоимость и т.п.);

– «Выбранный тип продукции или материал не существует» – возникает при попытке сохранить продукт с несуществующим идентификатором типа или материала;

– «Продукт не найден» – появляется при попытке редактирования, удаления или просмотра цехов несуществующего продукта;

– «Ошибка при сохранении: ...» / «Ошибка при удалении: ...» – общее сообщение об исключении при работе с БД, требует анализа логов или проверки подключения;

– «Не удалось выполнить расчёт. Проверьте правильность введённых данных» – возникает, если сервис расчёта вернул -1 (неверные идентификаторы или параметры).

При возникновении ошибок рекомендуется:

1) проверить строку подключения к БД;

2) убедиться, что все необходимые таблицы созданы и содержат данные;

3) проверить права доступа пользователя БД;

4) при необходимости включить подробные ошибки в среде разработки (отключить UseExceptionHandler в Program.cs).

2 Дипломный проект

2.1 Введение по дипломному проекту

В условиях повсеместной цифровизации компьютерная техника стала неотъемлемой частью инфраструктуры любого предприятия - от малого бизнеса до крупных логистических компаний. Бесперебойная работа компьютеров, серверов и периферийного оборудования критически важна для обеспечения непрерывности ключевых бизнес-процессов: документооборота, взаимодействия с клиентами, бухгалтерского учета и управления заказами. Это создает устойчивый спрос на услуги по ремонту и обслуживанию компьютерной техники.

Однако многие сервисные компании, оказывающие такие услуги, до сих пор используют устаревшие, разрозненные методы организации своей работы. Учет заявок ведется в бумажных журналах или электронных таблицах, общение с клиентами происходит по телефону и через мессенджеры, а информация о мастерах, запчастях и истории ремонтов хранится отдельно. Такой подход ведет к потере данных, трудностям в отслеживании статусов заказов, ошибкам при расчете стоимости и, как следствие, к снижению качества обслуживания и неудовлетворенности клиентов.

Целью данного курсового проекта является разработка серверной части веб-приложения для автоматизации управления сервисным центром по ремонту и обслуживанию компьютерной техники.

Задачи работы:

– провести анализ предметной области и существующих аналогов программных продуктов;

– сформулировать функциональные и нефункциональные требования к серверной части приложения;

– обосновать выбор инструментальных средств и технологий разработки;

– спроектировать архитектуру приложения, схему базы данных и API;

– реализовать основные модули серверной части на основе выбранной технологии;

– провести базовое тестирование работоспособности разработанного приложения.

2.2 Анализ предметной области

В современном мире информационные технологии играют ключевую роль в функционировании любой организации, включая предприятия сферы логистики и грузоперевозок, такие как ООО «ТК Делис Транс». Компания занимается организацией перевозок по России и странам СНГ, предоставляя широкий спектр транспортно-логистических услуг: автомобильные и железнодорожные перевозки, контейнерные перевозки, страхование грузов и оформление сопроводительных документов.

Однако, как и многие предприятия малого и среднего бизнеса, организация активно использует компьютерную технику для ведения бухгалтерии, документооборота, взаимодействия с клиентами и обработки заказов. С ростом объёма данных и информационных процессов возрастает потребность в качественном техническом обслуживании компьютерного оборудования, программного обеспечения и сетевой инфраструктуры

Общая характеристика предметной области:

Обслуживание и ремонт компьютерной техники - это неотъемлемая часть работы любой современной компании. Компьютеры, серверы, сетевые устройства, офисная техника и периферийное оборудование требуют регулярного технического обслуживания, своевременного ремонта и программной поддержки. От бесперебойной работы этой техники напрямую зависит эффективность всех бизнес-процессов.

На сегодняшний день большинство компаний обращаются в специализированные сервисные центры или частные фирмы для проведения профилактики, ремонта и настройки техники.

Такие услуги включают:

– диагностику и устранение неисправностей;

– установку и настройку операционных систем и программ;

– обслуживание сетевого оборудования;

– замену комплектующих;

– профилактику и модернизацию компьютерной техники.

Проблема заключается в том, что взаимодействие между клиентом и сервисной организацией зачастую осуществляется в устаревшей форме - по телефону, электронной почте или через мессенджеры. Это усложняет приём заказов, ведение учёта клиентов, отслеживание статуса заявок и формирование отчётности.

В результате возникают следующие трудности:

– отсутствие централизованного хранилища данных о клиентах и заказах;

– сложность контроля выполнения заявок и сроков ремонта;

– низкий уровень автоматизации обработки заказов;

– повышенная нагрузка на менеджеров и специалистов;

– неудобство для клиентов при обращении за услугами.

Необходимость разработки веб-приложения:

Для решения этих проблем целесообразно создать веб-приложение по предоставлению услуг ремонта и обслуживания компьютерной техники, которое обеспечит автоматизацию ключевых процессов.

Такое приложение позволит пользователям:

– быстро оформить заявку на ремонт или обслуживание оборудования через интернет;

– получить информацию о статусе выполнения заказа;

– просматривать историю обращений;

– получать уведомления о готовности техники и стоимости услуг.

Для компании внедрение подобного решения создаст ряд преимуществ:

– снижение нагрузки на сотрудников за счёт автоматизации приёма заказов;

– повышение прозрачности и управляемости процессов обслуживания;

– ускорение обработки заявок и обратной связи с клиентом;

– формирование базы данных клиентов и отчётности для анализа работы;

– улучшение конкурентоспособности на рынке.

Проблематика существующих решений:

Большинство существующих систем для сервисных центров либо избыточно сложны и ориентированы на крупные организации, либо, наоборот, слишком просты и не обеспечивают необходимых функций. Нередко компании используют разные несвязанные инструменты: Excel-таблицы, телефонные заявки, почтовые переписки. Это приводит к дублированию информации и потере данных.

Кроме того, многие старые подходы не соответствуют современным требованиям клиентов, которые ожидают онлайн-сервис с понятным интерфейсом, возможностью отслеживания заказа и обратной связи. Создание клиентской части веб-приложения решает эти задачи, предоставляя пользователю удобный доступ к услугам компании и упрощая взаимодействие с сервисом.

2.3 Требования к программному продукту

Функциональные требования программы является управление заявками на ремонт. Серверная часть должна предоставлять интерфейсы (API) для создания новых заявок, их просмотра, редактирования и обновления статуса. Статусы должны отражать типичный жизненный цикл заказа: «Принята», «На диагностике», «В ремонте», «Готов к выдаче», «Выдана».

Для работы с данными должна быть реализована функциональность по управлению клиентами и учетными записями сотрудников. Это включает аутентификацию и авторизацию пользователей, разделяя права доступа для клиентов, менеджеров и инженеров. Менеджеры должны иметь полный доступ к заявкам, в то время как клиенты - только к своим.

Система должна обеспечивать базовое ведение справочников, таких как типы неисправностей и виды обслуживаемой техники, что упростит процесс оформления заявки и последующей аналитики.

Нефункциональные требования включают в себя производительность и масштабируемость являются важными аспектами. Серверная часть должна эффективно обрабатывать базовые нагрузки, характерные для малого бизнеса. Архитектура должна быть спроектирована таким образом, чтобы позволять добавление нового функционала в будущем без кардинальных переделок.

Безопасность данных является обязательным условием. Все пароли должны храниться в базе данных в хешированном виде. Необходимо реализовать защиту от основных веб-угроз, а также валидацию всех входящих данных на стороне сервера.

Требуется обеспечить надежное взаимодействие с базой данных SQL Server. Код доступа к данным должен быть оптимизирован, использовать параметризованные запросы для защиты от SQL-инъекций и обеспечивать целостность информации.

Эксплуатационные требования - серверная часть будет реализована как веб-приложение на платформе ASP.NET, развертываемое на сервере под управлением операционной системы Windows. В качестве сервера базы данных будет использоваться Microsoft SQL Server.

Минимальные системные требования для запуска серверной части:

- операционная система Windows 10 / Windows Server 2012 R2 и выше;
- установленный .NET 8.0 Runtime (или .NET 6.0 LTS);
- система управления базами данных microsoft SQL Server 2019 Express Edition и выше;
- процессор 2-ядерный процессор с тактовой частотой от 2 ГГц;
- не менее 4 ГБ ОЗУ;
- не менее 2 ГБ для установки необходимых компонентов и базы данных.

Логика приложения будет разработана на языке C# с использованием подхода MVC (Model-View-Controller) или Web API для четкого разделения ответственности компонентов. Для доступа к данным планируется использовать Entity Framework Core, что ускорит разработку и обеспечит удобную работу с объектной моделью.

Система должна быть готова к развертыванию в среде Visual Studio 2022 и сопровождаться документацией, достаточной для ее запуска и дальнейшей поддержки.

2.4 Обоснование выбора инструментальных средств разработки

Выбор технологического стека для серверной части веб-приложения основан на комплексном анализе требований проекта, производительности технологий и эффективности процесса разработки. Рассматривались различные

альтернативы, однако окончательный выбор остановлен на связке C#, ASP.NET Core и SQL Server.

Сравнение серверных платформ рассматривало альтернативы в лице Java с фреймворком Spring и Python с Django. Java предлагает схожий уровень производительности и строгой типизации, однако обладает более сложной конфигурацией и требователен к ресурсам. Python обеспечивает высокую скорость разработки, но динамическая типизация языка может приводить к ошибкам в рантайме, что нежелательно для проектов с требованием к надежности. C# и ASP.NET Core представляют оптимальный баланс - современный язык со строгой типизацией, высокая производительность, близкая к C++, и при этом относительно низкий порог входа по сравнению с Java.

Анализ систем управления базами данных включал сравнение Microsoft SQL Server с PostgreSQL и MySQL. PostgreSQL является мощной открытой СУБД с богатой функциональностью, однако требует дополнительных усилий по настройке в среде Windows. MySQL, хотя и популярен для веб-приложений, уступает в производительности сложных запросов и возможностях оптимизации. SQL Server выбран благодаря тесной интеграции с экосистемой Microsoft, превосходным инструментам анализа запросов и возможности использования бесплатной Express-версии для учебного проекта.

Среда разработки рассматривалась в контексте Visual Studio 2022 против JetBrains Rider и Visual Studio Code. Rider предлагает отличную функциональность, но является платным решением. Visual Studio Code - легковесный редактор, но требует значительной донастройки для полноценной работы с .NET. Visual Studio 2022 предоставляет полную интегрированную среду «из коробки» с мощным отладчиком, профилировщиком и встроенной поддержкой SQL Server, что критически важно для учебного проекта с ограниченными сроками.

Дополнительные технологии включают Entity Framework Core как основной ORM. Рассматривалась альтернатива в виде Dapper - более легковесного микрORM, однако для учебного проекта важнее возможность быстрого прототипирования и автоматической генерации миграций базы данных, что обеспечивает Entity Framework Core.

Выбранный стек технологий образует сбалансированное решение, где все компоненты оптимально интегрированы между собой, что обеспечивает предсказуемую производительность, сокращает время разработки и соответствует как учебным целям проекта, так и потенциальным требованиям промышленной эксплуатации.

2.5 Проектирование и разработка базы данных

После завершения проектирования логической и физической модели данных был осуществлен практический этап создания базы данных. На основании

разработанной ERD-диаграммы и спецификаций таблиц были сформированы SQL-скрипты, реализующие структуру базы данных в среде Microsoft SQL Server.

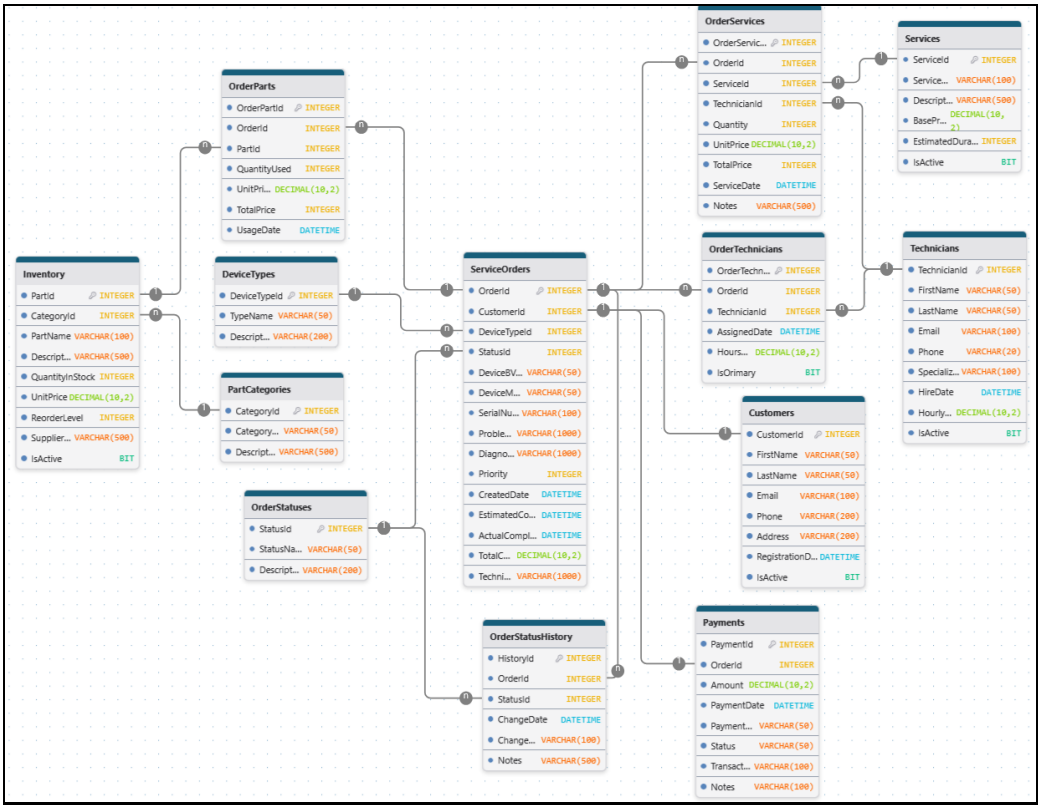


Рисунок 2.1 – ERD-диаграмма базы данных

Была создана таблица OrderStatuses, являющаяся справочником и определяющая жизненный цикл заявки. Ее наличие обеспечивает целостность данных и упрощает изменение списка статусов в системе:

– новая;
– диагностика;
– в ремонте;
– готово;
– выдано.

Таблица DeviceTypes содержит классификацию обслуживаемой техники, что позволяет структурировать заявки и проводить аналитику по типам устройств.В нее внесены основные категории:

– ноутбуки;
– стационарные ПК;
– серверное оборудование;
– принтеры и МФУ.

Центральной сущностью системы является таблица ServiceOrders, которая хранит полную информацию о каждой заявке на ремонт. Структура записи включает:

– ссылки на клиента и тип устройства;
– детали проблемы и диагноз;
– текущий статус и приоритет;
– даты создания, планируемого и фактического завершения;
– расчетную итоговую стоимость.

Для работы с клиентами и мастерами предназначены таблицы Customers и Technicians. Их структура обеспечивает хранение контактных данных, информации для связи и отслеживания активности. Таблица Services содержит каталог оказываемых услуг с базовой стоимостью и нормативом времени, а связующая таблица OrderServices фиксирует факт выполнения конкретной услуги в рамках заказа.

Учет расходных материалов и комплектующих организован с помощью таблиц PartCategories, Inventory и OrderParts. Это позволяет контролировать остатки на складе, автоматически списывать использованные запчасти и корректно учитывать их стоимость в каждом заказе.

Таблица Inventory для каждой позиции хранит:
– название и описание;
– категорию;
– текущее количество и порог повторного заказа;
– закупочную цену.

Для отражения истории изменений и финансовых операций созданы таблицы OrderStatusHistory и Payments. Первая фиксирует все переходы заявки между статусами с указанием времени и автора изменения, вторая — хранит данные о произведенных клиентом оплатах.

Оптимизация производительности запросов обеспечена созданием индексов по ключевым полям, участвующим в фильтрации и соединениях, таким как CustomerId, StatusId в таблице заказов и OrderId в связанных таблицах. Дополнительно созданы представления (Views) для упрощения формирования отчетов: ActiveOrders для списка текущих работ, TechnicianWorkload для анализа загрузки мастеров и LowStockAlert для мониторинга критичного уровня запасов на складе.

2.6 Проектирование и реализация архитектуры Web API

Основой серверной части разрабатываемого приложения выступает Web API, построенное по архитектурному стилю REST. Данный подход был выбран в связи с его универсальностью, простотой интеграции и ориентацией на ресурсы, что идеально соответствует предметной области системы управления заявками. Каждая сущность — заказ, клиент, мастер — представляется в API как уникальный ресурс, доступ к которому осуществляется через стандартные HTTP-методы, что интуитивно понятно как для разработчиков, так и для потенциальных потребителей API.

Архитектура приложения реализована по принципу многослойной (n-tier) архитектуры с четким разделением ответственности между компонентами. Это обеспечивает высокую сопровождаемость, тестируемость и возможность независимого развития отдельных частей системы. Основными слоями являются:

– слой представления данных (Controllers), отвечающий за обработку HTTP-запросов и формирование ответов;

– слой бизнес-логики (Services), инкапсулирующий ключевые правила и процессы предметной области;

– слой доступа к данным (Data Access Layer), обеспечивающий абстракцию для работы с базой данных.

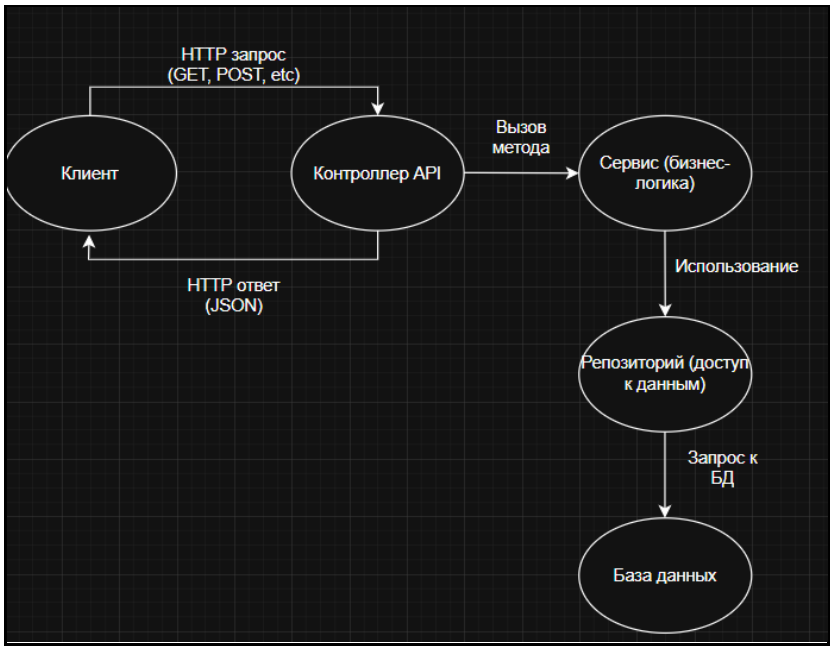


Рисунок 2.2 – Архитектура серверной части приложения

Контроллеры являются точками входа для обработки HTTP-запросов. Каждый контроллер отвечает за работу с одной ключевой сущностью системы.

Например, OrdersController обрабатывает все запросы, связанные с заявками на ремонт, по адресам, начинающимся с /api/orders.

Контроллеры реализуют стандартные операции CRUD и специфические бизнес-операции. Типичная структура контроллера включает:

– метод Get() для получения списка заявок с поддержкой фильтрации по статусу, клиенту или дате;

– метод Get(int id) для получения детальной информации о конкретной заявке;

– метод Post([FromBody] CreateOrderRequest request) для создания новой заявки с валидацией входящих данных;

– метод Put(int id, [FromBody] UpdateOrderRequest request) для обновления информации о заявке;

– метод Patch(int id, [FromBody] UpdateOrderStatusRequest request) для изменения статуса заявки.

Бизнес-логика приложения сосредоточена в сервисном слое (Services), который реализует сложные процессы обработки данных. Ключевыми бизнес-процессами системы являются:

Создание и обработка заявки на ремонт. При создании новой заявки система выполняет следующие действия:

– валидация данных клиента и устройства;

– автоматическое определение типа устройства на основе введенной модели;

– расчет предварительной стоимости на основе выбранных услуг и типовых расценок;

– назначение приоритета заявки в зависимости от критичности проблемы.

Управление статусами заявок. Система реализует строгий жизненный цикл заявки с контролем допустимых переходов между статусами. Например, переход из статуса «Диагностика» в статус «В ремонте» возможен только после того, как мастер заполнил диагностические заметки и подтвердил необходимость ремонта. Каждое изменение статуса автоматически фиксируется в таблице OrderStatusHistory для полного аудита.

Назначение мастеров и расчет трудозатрат. При назначении мастера на заявку система учитывает:

– специализацию мастера (работа с ноутбуками, серверами и т.д.);

– текущую загрузку мастера на основе активных заявок;

– приоритет заявки.

Система автоматически рассчитывает затраченное время и формирует данные для расчета заработной платы.

Управление складом и учет запчастей. При использовании запчастей в ремонте система:

– проверяет наличие необходимого количества на складе;

– резервирует запчасти для текущей заявки;

– автоматически списывает использованные компоненты;

– генерирует предупреждения при достижении минимального остатка.

Сервисный слой работает с моделями предметной области (Domain Models) и Data Transfer Objects (DTO). Модели предметной области (например, класс ServiceOrder) отражают внутреннюю структуру бизнес-сущностей со всеми их поведением и инвариантами. DTO (например, OrderDetailsDto или CreateOrderRequest) — это упрощенные классы, предназначенные исключительно для передачи данных между слоями и клиентом; они содержат только необходимые свойства, что повышает безопасность и эффективность сетевого обмена.

Слой доступа к данным построен с использованием паттерна Repository, что позволяет абстрагировать логику работы с базой данных. Для каждой агрегатной сущности создается свой репозиторий (например, IOrderRepository), который определяет контракт для операций с данными: получение по ID, добавление, фильтрация по критериям. Реализация репозитория (OrderRepository) использует Entity Framework Core и DbSet<T> для выполнения этих операций. Это скрывает сложность LINQ-запросов и специфику ORM от вышележащих слоев, облегчает модульное тестирование бизнес-логики путем подмены репозитория mock-объектом и централизует все взаимодействие с контекстом базы данных (AppDbContext).

Все компоненты связываются с помощью механизма Dependency Injection (DI), встроенного в ASP.NET Core. Каждая зависимость регистрируется в контейнере служб в файле Program.cs. Контроллеры и сервисы получают необходимые экземпляры через конструктор, что обеспечивает слабую связанность компонентов: каждый модуль зависит от абстракций (интерфейсов), а не от конкретных реализаций.

Выбранная архитектура обеспечивает четкое разделение ответственности, высокую степень абстракции в работе с данными, безопасность и удобство сопровождения кода. Она создает надежный фундамент для реализации всего запланированного функционала и для будущего расширения системы, например, добавления модуля отчетности или интеграции со сторонними сервисами отправки уведомлений.

2.7 Инструкция по работе с веб приложением

Интерфейс формы разделен на две логические вкладки: «Регистрация» и «Вход». В режиме регистрации пользователь заполняет обязательные поля: имя, фамилия, адрес электронной почты (используется как логин), номер телефона и пароль. Система в реальном времени проводит базовую валидацию данных, проверяя корректность email, сложность пароля и уникальность адреса электронной почты в системе.

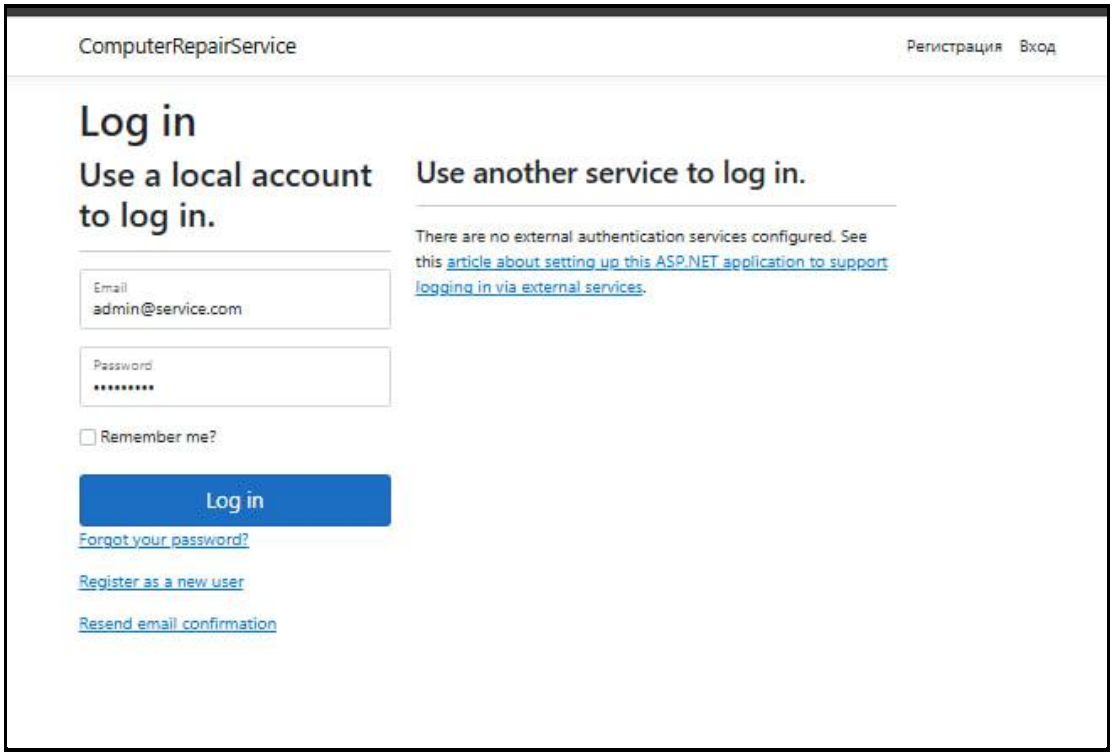


Рисунок 2.3 – Форма регистрации и входа в систему

В личном кабинете администратора реализована отдельная панель управления ролями, где можно просматривать список всех пользователей, назначать им роли (Клиент, Работник, Администратор) и управлять дополнительными разрешениями. Изменения вступают в силу немедленно, так как при каждом запросе система заново проверяет актуальные права пользователя.

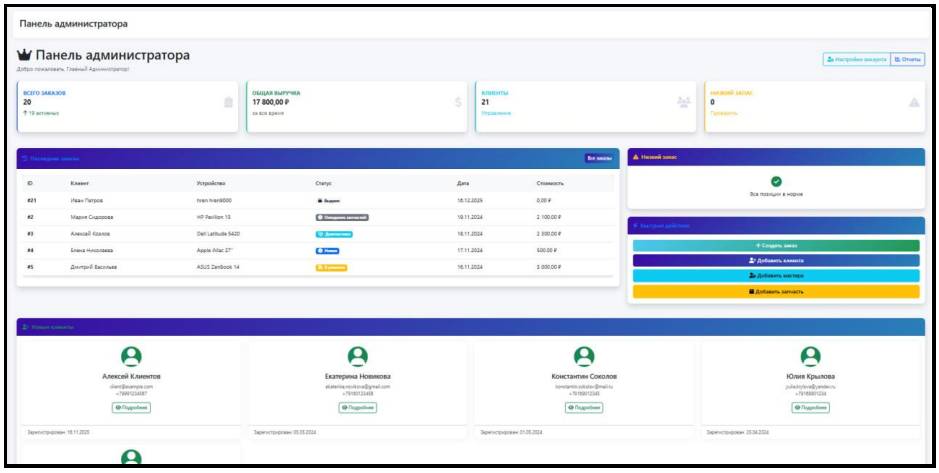


Рисунок 2.4 – Панель администратора

Для создания новой заявки на ремонт техники пользователю необходимо перейти в раздел «Личный кабинет». На данной странице отображается сводная информация о статусе обращений: количество активных и завершенных заявок, а также общая сумма потраченных средств. Перед оформлением заказа рекомендуется проверить актуальность контактных данных в блоке «Мой профиль» (телефон и email), так как они используются для связи с мастером. Для создания нового обращения следует нажать на кнопку «+ Создать заказ» в нижней части экрана, после чего откроется форма заполнения деталей поломки.

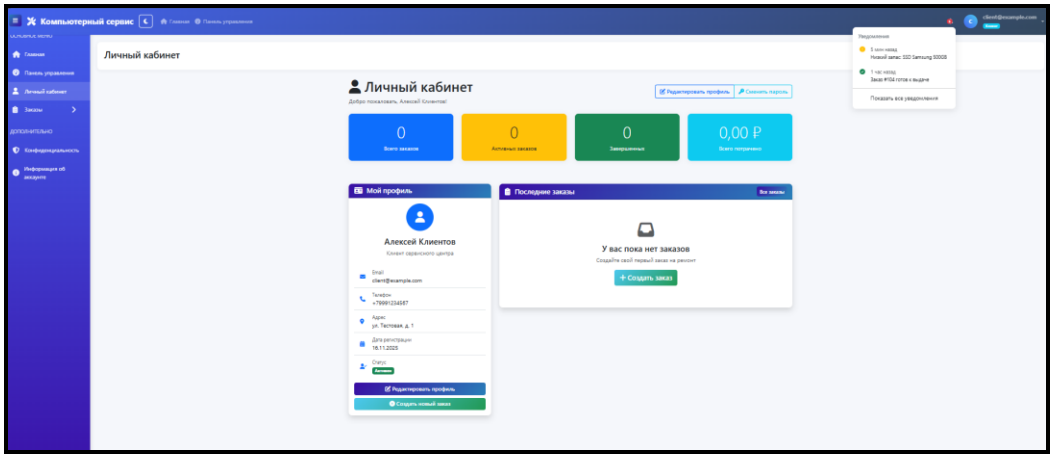


Рисунок 2.5 – Личный кабинет пользователя

При входе в систему сотрудник сервисного центра попадает на рабочую панель управления. В верхнем блоке отображается текущая загрузка: количество активных задач и завершенных ремонтов в текущем месяце, что позволяет мастеру планировать рабочее время. Для навигации по функциям системы используется боковое меню. Раздел «Заказы» предназначен для просмотра списка поступивших клиентских заявок, раздел «Склад» - для учета комплектующих. Чтобы изменить свою специализацию или контактный номер, сотруднику необходимо нажать кнопку «Редактировать профиль» и внести изменения в соответствующие поля.

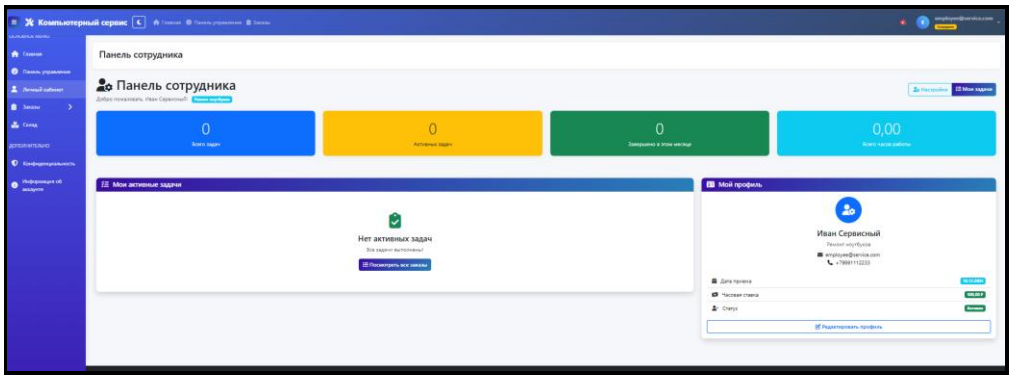


Рисунок 2.6 – Личный кабинет работника

Главная страница системы представляет собой удобную панель управления, содержащую ключевые разделы и инструменты для быстрого доступа к основным функциям. В верхней части размещено главное меню, позволяющее перейти к разделам клиентов, заказов, мастеров, складского учёта и другим модулям системы. Центральный блок интерфейса включает карточки с основными показателями.

В нижней части страницы отображается таблица «Последние заказы», содержащая информацию о недавних обращениях: данные клиента, описание проблемы, текущий статус и дату создания заявки.

Таблица помогает пользователю быстро ориентироваться в текущих задачах и своевременно реагировать на новые обращения.

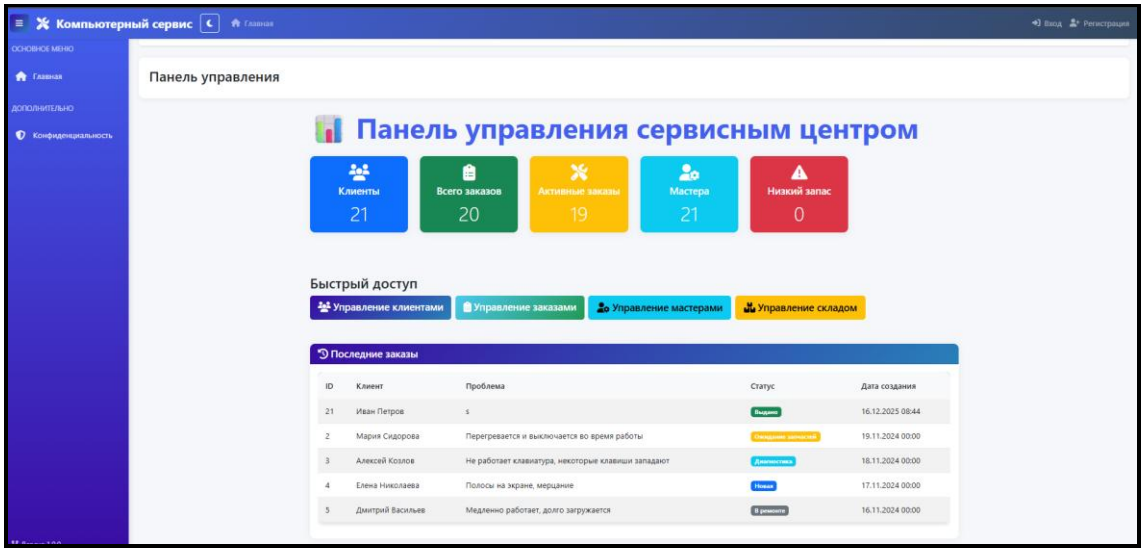


Рисунок 2.7 – Главная страница

Страница «Клиенты» предназначена для управления клиентской базой сервисного центра. В верхней части расположена панель быстрого доступа, включающая кнопку «Добавить клиента», которая позволяет оперативно создать новую запись, указав основные контактные данные. Присутствует возможность сделать поиск по имени, фамилии email, телефону.

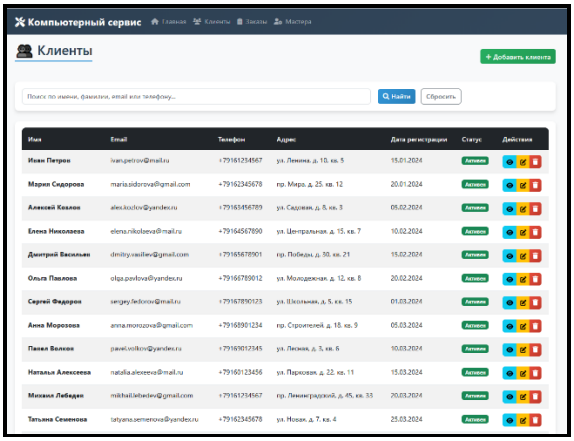


Рисунок 2.8 – Страница управления клиентами

При нажатии на «синий глаз» справа от информации о клиенте, открывается детальная информация о клиенте.

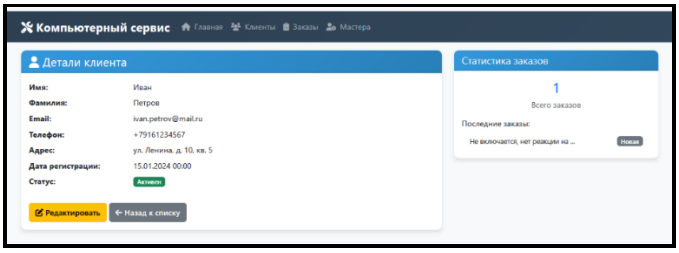


Рисунок 2.9 – Детальная информация о клиенте

При нажатии на кнопку редактировать открывается окно редактирования клиента.

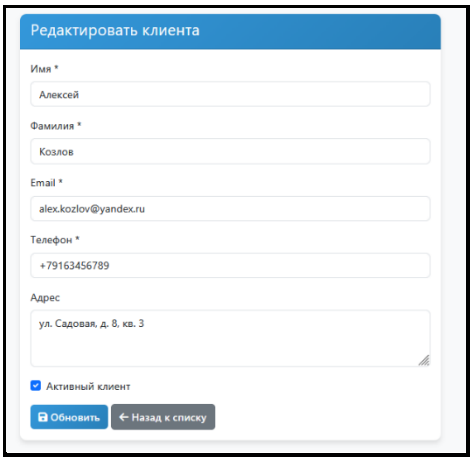


Рисунок 2.10 – Редактор информации о клиенте

При нажатии на кнопку «Клиенты» открывается страница, где отображены все заказы.

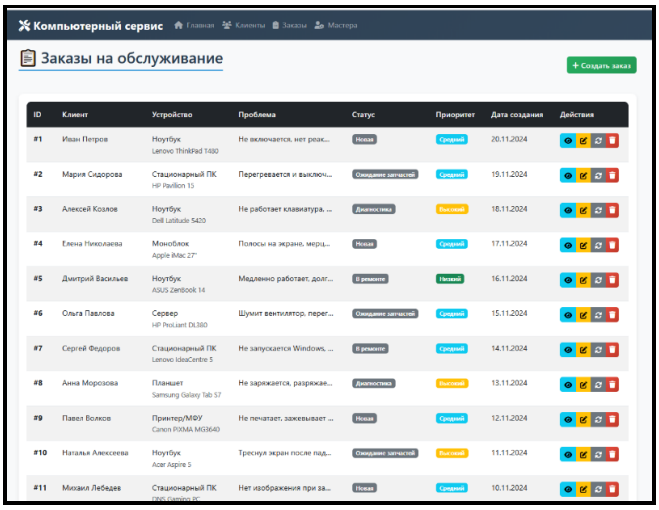


Рисунок 2.11 – Страница управления заказами

Нажав на синюю кнопку откроется страница с деталями заказа, где можно увидеть всю настоящую информацию о нем, а также редактировать, изменить его статус.

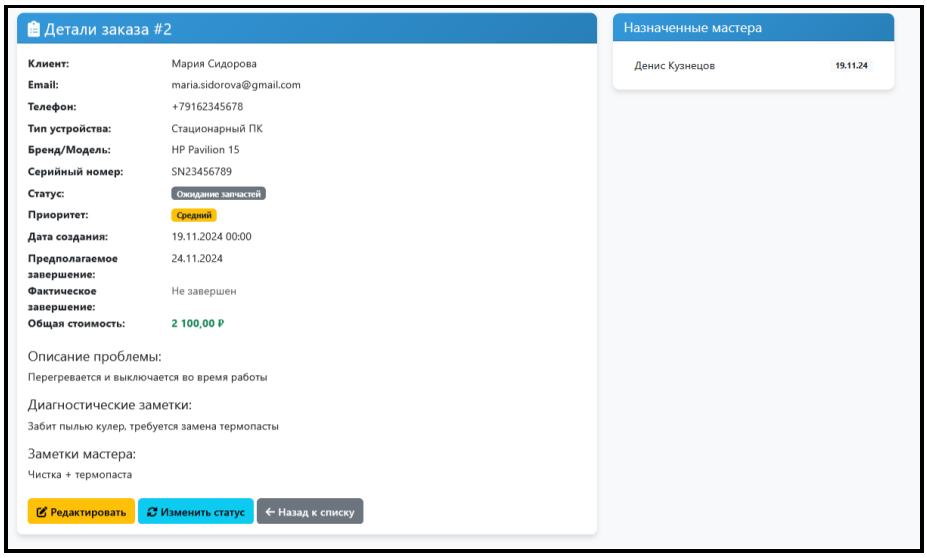


Рисунок 2.12 – Детали заказа

Нажав на оранжевую кнопку откроется страница с редактированием заказа, где можно изменить любую интересующую информацию.

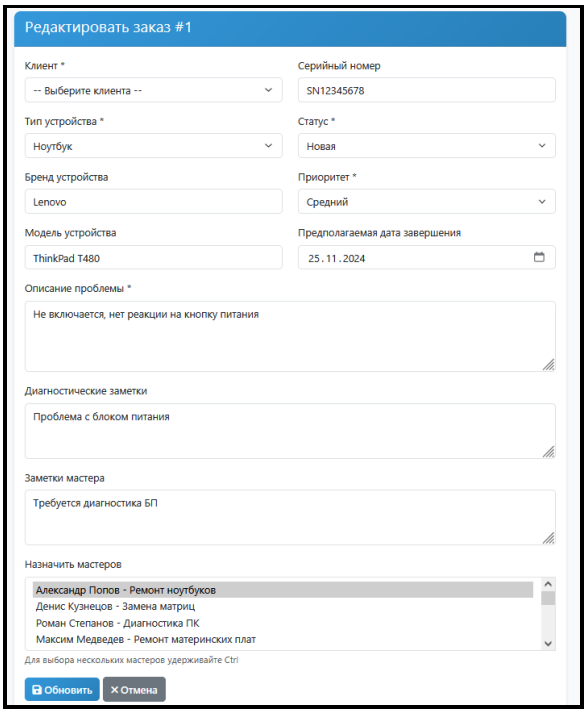


Рисунок 2.13 – Страница редактирования заказа

Нажав на серую кнопку откроется страница с изменением статуса заказа.

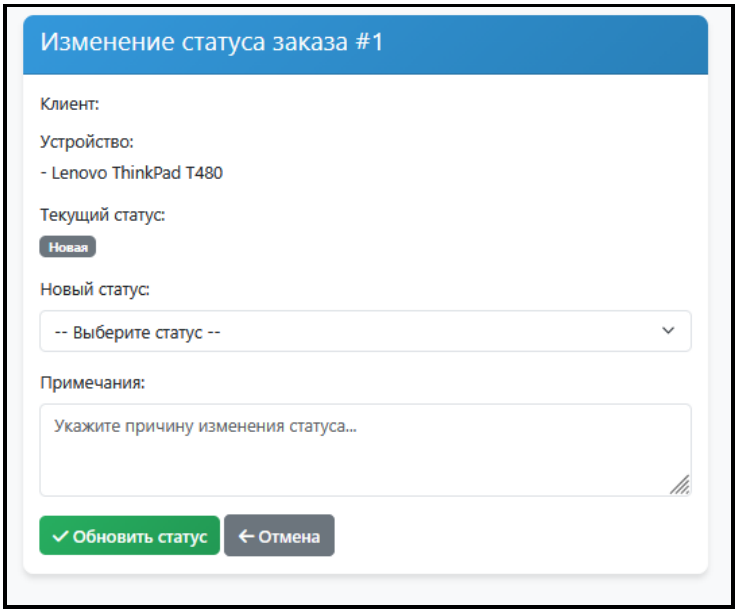


Рисунок 2.14 – Страница редактирования заказа

Нажав на кнопку «Создать заказ» откроется страница с созданием заказа.

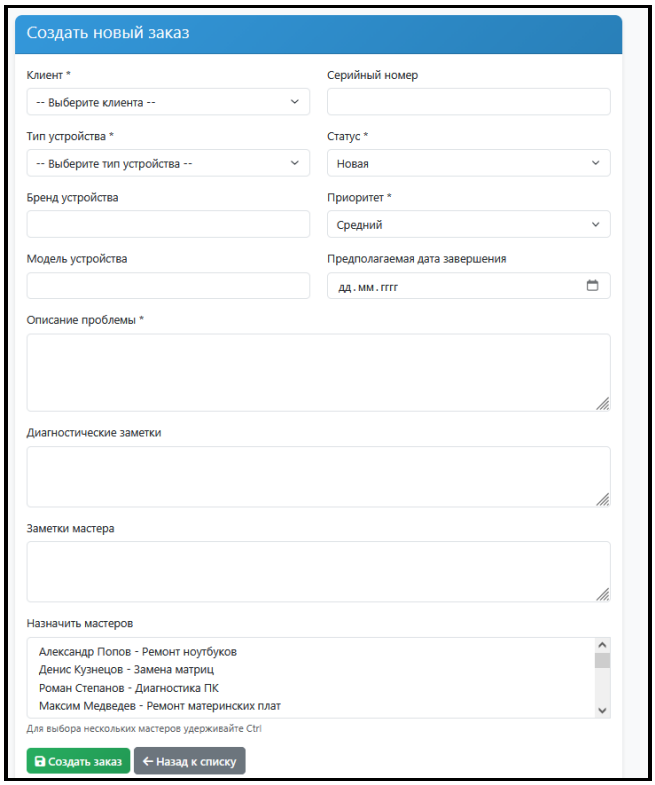


Рисунок 2.15 – Страница редактирования заказа

Нажав на кнопку «Мастера» откроется страница с управлением мастеров.

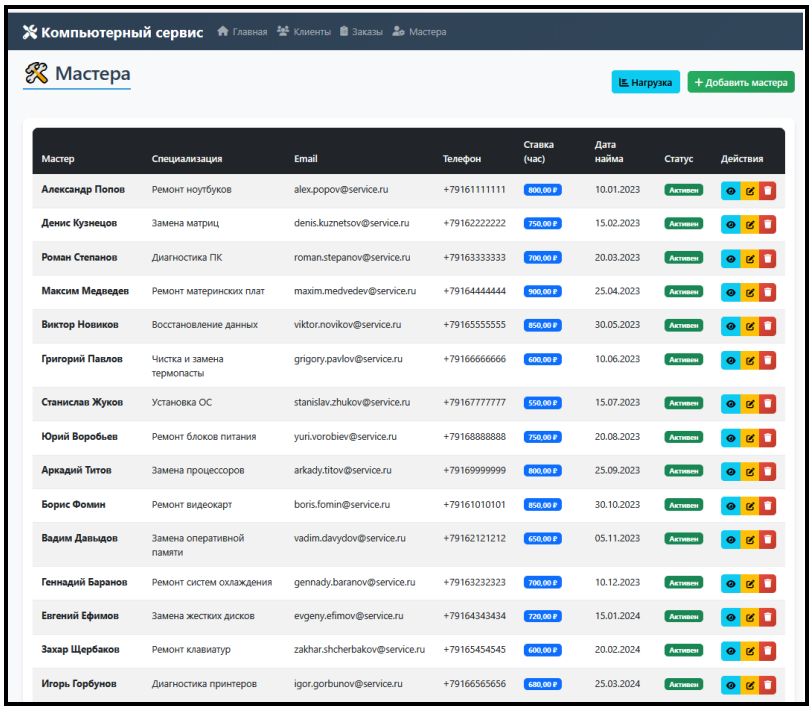


Рисунок 2.16 – Страница мониторинга мастеров

При нажатии на синюю кнопку, откроется страница с детальной информацией о мастере.

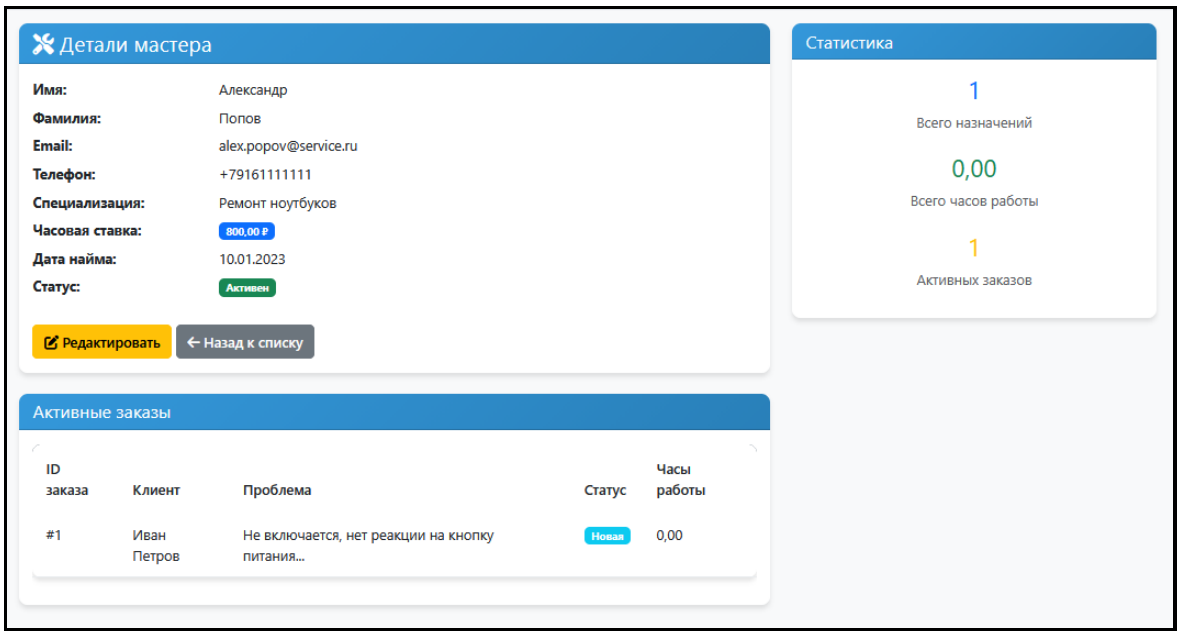


Рисунок 2.17 – Страница информации о мастере

При нажатии на оранжевую кнопку, откроется страница с редактированием информации о мастере.

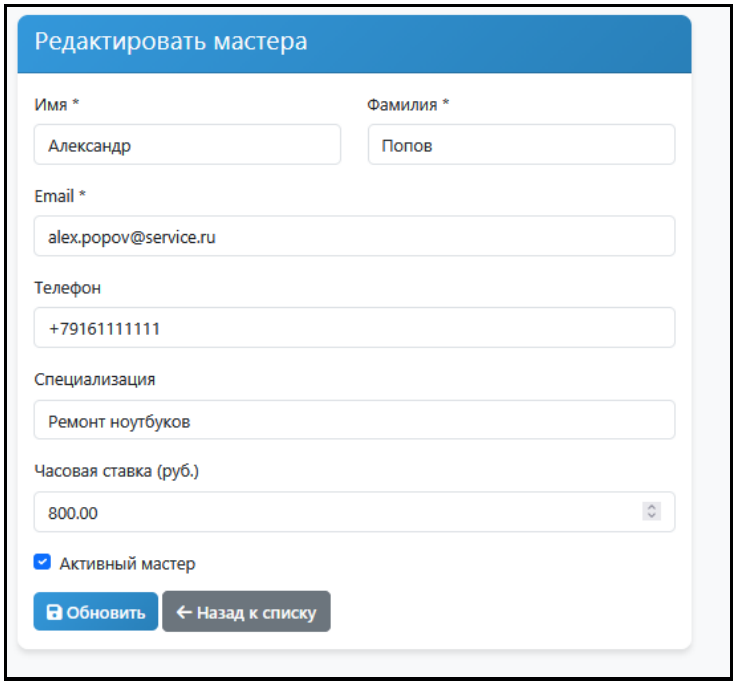


Рисунок 2.18 – Страница редактирования информации о мастере

При нажатии на кнопку «Добавить мастера», откроется страница с возможностью добавления нового мастера.

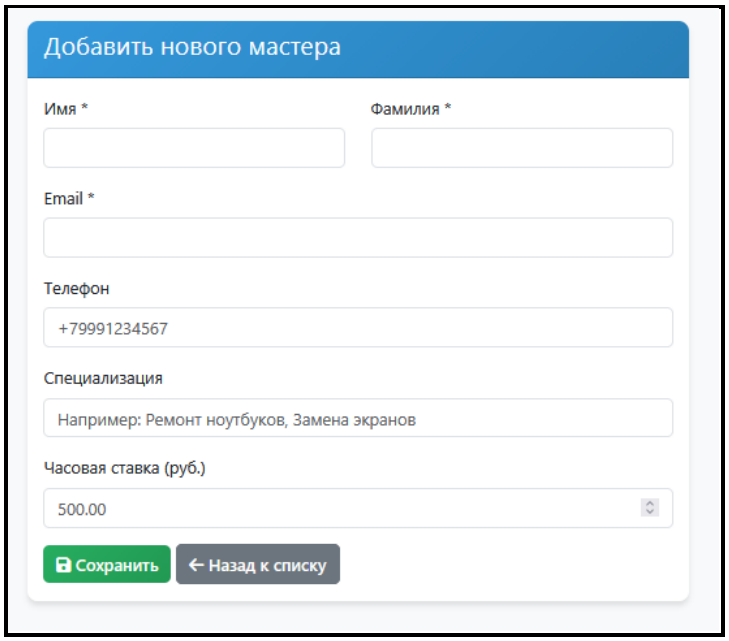


Рисунок 2.19 – Страница добавления мастера

При нажатии на кнопку «Нагрузка» в правой верхней части сайта, откроется страница с информацией о нагрузке мастеров.

Нагрузка мастеров

← Назад к списку

Статистика нагрузки мастеров

Мастер	Специализация	Активных заказов	Всего часов работы	Статус нагрузки	Примерная стоимость работ
Виктор Новиков viktor.novikov@service.ru	Восстановление данных	1	2,0 ч	Низкая	1 700,00 ₽
Денис Кузнецов denis.kuznetsov@service.ru	Замена матриц	1	1,5 ч	Низкая	1 125,00 ₽
Геннадий Баранов gennady.baranov@service.ru	Ремонт систем охлаждения	1	1,5 ч	Низкая	1 050,00 ₽
Леонид Карпов leonid.karpov@service.ru	Ремонт разъемов	1	1,0 ч	Низкая	740,00 ₽
Станислав Жуков stanislav.zhukov@service.ru	Установка ОС	1	1,0 ч	Низкая	550,00 ₽
Аркадий Титов arkady.titov@service.ru	Замена процессоров	1	0,5 ч	Низкая	400,00 ₽
Петр Маслов petr.maslov@service.ru	Обслуживание серверов	1	0,5 ч	Низкая	475,00 ₽
Александр Попов alex.popov@service.ru	Ремонт ноутбуков	1	0,0 ч	Низкая	—
Никита Афанасьев nikita.afanasev@service.ru	Чистка от пыли	1	0,0 ч	Низкая	—
Олег Власов oleg.vlasov@service.ru	Замена кулеров	1	0,0 ч	Низкая	—
Евгений Ефимов evgeny.efimov@service.ru	Замена жестких дисков	1	0,0 ч	Низкая	—
Захар Щербаков zakhar.shcherbakov@service.ru	Ремонт клавиатур	1	0,0 ч	Низкая	—

Рисунок 2.20 – Страница мониторинга нагрузки мастеров

На главной странице, нажав на кнопку «Управление складом» откроется страница с мониторингом запчастей на складе.

Склад запчастей

Название	Категория	Количество на складе	Цена за единицу	Минимальный запас	Статус
SSD Samsung 870 Evo 500GB	Жесткие диски	15	4 500,00 ₽	3	В норме
HDD Seagate Barracuda 1TB	Жесткие диски	8	3 500,00 ₽	2	В норме
Оперативная память Kingston 8GB DDR4	Оперативная память	25	2 500,00 ₽	5	В норме
Оперативная память Corsair 16GB DDR4	Оперативная память	12	4 800,00 ₽	3	В норме
Процессор Intel Core i5-12400	Процессоры	6	18 500,00 ₽	2	В норме
Процессор AMD Ryzen 5 5600X	Процессоры	4	16 800,00 ₽	1	В норме
Материнская плата ASUS PRIME B660	Материнские платы	7	8 900,00 ₽	2	В норме
Материнская плата Gigabyte B550	Материнские платы	5	10 200,00 ₽	2	В норме
Блок питания DeepCool 750W	Блоки питания	10	5 200,00 ₽	3	В норме
Блок питания Cooler Master 650W	Блоки питания	6	7 800,00 ₽	2	В норме
Кулер для процессора DeepCool GAMMAXX	Системы охлаждения	18	1 800,00 ₽	5	В норме
Термопаста Arctic MX-4	Системы охлаждения	30	450,00 ₽	10	В норме
Батарея для ноутбука Dell 45Wh	Батареи	9	3 200,00 ₽	3	В норме
Батарея для ноутбука HP 42Wh	Батареи	7	2 900,00 ₽	2	В норме
Матрица для ноутбука 15.6" FullHD	Экранные модули	11	6 500,00 ₽	4	В норме
Матрица для ноутбука 14" HD	Экранные модули	8	4 200,00 ₽	3	В норме
Клавиатура для ноутбука Lenovo	Клавиатуры	14	1 800,00 ₽	5	В норме
Клавиатура для ноутбука ASUS	Клавиатуры	12	1 600,00 ₽	4	В норме
Вентилятор кулера 120mm	Системы охлаждения	22	600,00 ₽	8	В норме

Рисунок 2.21 – Страница мониторинга запчастей

Заключение

В ходе выполнения задания учебной практики была разработана информационная система «ComfortFurniture» для учёта продукции мебельной компании.

Спроектирована база данных в третьей нормальной форме, включающая таблицы ProductTypes, MaterialTypes, Workshops, Products и ProductWorkshops. Выполнена загрузка тестовых данных из Excel-файлов, сгенерирована ER-диаграмма.

Разработано веб-приложение на ASP.NET Core MVC с использованием Entity Framework Core. Реализован функционал просмотра, добавления, редактирования и удаления продукции, просмотра цехов для конкретного продукта, а также расчёта сырья с учётом коэффициентов и процента потерь.

Интерфейс выполнен в соответствии с руководством по стилю: шрифт Candara, цветовая гамма #FFFFFF, #D2DFFF, #355CBD, добавлены логотип и иконка приложения.

Проведено тестирование (20 проверок), все тесты выполнены успешно, аварийных завершений не зафиксировано. Задания всех модулей выполнены в полном объёме.

Опыт, полученный при выполнении учебной практики был применён для усовершенствования дипломного проекта: оптимизирована структура базы данных, улучшена обработка исключений, реализована валидация данных, доработана навигация и приведён к единому стилю интерфейс.

Разработанный программный продукт обеспечивает удобную работу с заявками, легко масштабируется и может дополняться новым функционалом. Проект полностью соответствует поставленным задачам и представляет собой эффективный инструмент для сервисных центров.В ходе выполнения учебной практики были выполнены все поставленные задачи. Главным итогом работы стала полученная программа и документация к варианту демонстрационного экзамена, а также реализация практического модуля к дипломному проекту.

Для достижения этого результата, были выполнены все этапы разработки:

– проектировка базы данных и написание кода;

– настройка безопасность и роли;

– создание интерфейса удобным и защищенным;

– оформил документацию.

Весь цикл работы сохранен в системе контроля версий Git. Созданные программные продукты работают стабильно и решают задачи организаций.

Библиография

1 Основы веб-технологий для бизнес-приложений / Д.А. Морозов. – М.: Интернет-университет информационных технологий, 2023. – 280 с.

2 JavaScript и современные фреймворки для создания интерактивных интерфейсов / К.П. Васильев. – СПб.: Питер, 2022. – 416 с.

3 Базы данных и SQL: проектирование и применение / Л.М. Федорова. – М.: ДМК Пресс, 2023. – 264 с.

4 Безопасность web-приложений: теория и практика / С.И. Павлов. – М.: Лаборатория знаний, 2022. – 183 с.

5 HTML5 и CSS3 в разработке современных веб-приложений / М.А. Семенова. – М.: Эксмо, 2023. – 320 с.

Приложение А
(справочная)
Листинг программы

SQL запрос для создания БД

```
CREATE DATABASE ComfortFurnitureDB;
GO

USE ComfortFurnitureDB;
GO

-- Ñïðàâî÷íèê òèïîâ ïðîäóêöèè
CREATE TABLE ProductTypes (
    ProductTypeId INT IDENTITY(1,1) PRIMARY KEY,
    Name NVARCHAR(100) NOT NULL,
    Coefficient FLOAT NOT
);
GO

-- Ñïðàâî÷íèê òèïîâ ìàòåðèàëîâ
CREATE TABLE MaterialTypes (
    MaterialTypeId INT IDENTITY(1,1) PRIMARY KEY,
    Name NVARCHAR(100) NOT NULL,
    LossPercent FLOAT NOT NULL
);
GO

-- Öåõà
CREATE TABLE Workshops (
    WorkshopId INT IDENTITY(1,1) PRIMARY KEY,
    Name NVARCHAR(100) NOT NULL,
    Type NVARCHAR(50) NOT NULL,
    EmployeeCount INT NOT NULL
);
GO

-- Îñíîâíàÿ òàáëèöà ïðîäóêöèè
CREATE TABLE Products (
    ProductId INT IDENTITY(1,1) PRIMARY KEY,
    ProductTypeId INT NOT NULL FOREIGN KEY REFERENCES
ProductTypes(ProductTypeId),
    Name NVARCHAR(200) NOT NULL,
    Article NVARCHAR(50) NOT NULL UNIQUE,
    MinPriceForPartner DECIMAL(18,2) NOT NULL,
    MainMaterialId INT NOT NULL FOREIGN KEY REFERENCES
MaterialTypes(MaterialTypeId),

    Description NVARCHAR(MAX) NULL,
    Image VARBINARY(MAX) NULL
);
```

```
GO

CREATE TABLE ProductWorkshops (
    ProductWorkshopId INT IDENTITY(1,1) PRIMARY KEY,
    ProductId   INT   NOT   NULL   FOREIGN   KEY   REFERENCES
Products(ProductId) ON DELETE CASCADE,
    WorkshopId   INT   NOT   NULL   FOREIGN   KEY   REFERENCES
Workshops(WorkshopId) ON DELETE CASCADE,
    TimeHours FLOAT NOT NULL
);
GO

-- Èíäåêñû äëÿ óñêîðåíèÿ
CREATE INDEX IX_Products_Article ON Products(Article);
CREATE       INDEX       IX_ProductWorkshops_ProductId       ON
ProductWorkshops(ProductId);
```

ProductsController.cs

```
using Gusev284_UP.Data;
using Gusev284_UP.Models;
using Gusev284_UP.Services;      // для IRawMaterialCalculator
using Gusev284_UP.ViewModels;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Rendering;
using Microsoft.EntityFrameworkCore;

namespace Gusev284_UP.Controllers
{
    public class ProductsController : Controller
    {
        private readonly ApplicationDbContext _context;
        private readonly IRawMaterialCalculator _calculator;

        public ProductsController(ApplicationDbContext context,
IRawMaterialCalculator calculator)
        {
            _context = context;
            _calculator = calculator;
        }

        // ==================== МОДУЛЬ 2: Главная страница со
списком ====================
        public async Task<IActionResult> Index()
        {
            var products = await _context.Products
                .Include(p => p.ProductType)
                .Include(p => p.ProductWorkshops)
                .Select(p => new ProductIndexViewModel
                {
                    ProductId = p.ProductId,
```

```
                            Article = p.Article,
                            ProductTypeName = p.ProductType.Name,
                            Name = p.Name,
                            MinPriceForPartner = p.MinPriceForPartner,
                            TotalTimeHours                              =
(int)Math.Ceiling(p.ProductWorkshops.Sum(pw => pw.TimeHours))
                        })
                        .ToListAsync();

                    // Загружаем справочники для выпадающих списков в
модальном окне
                    ViewBag.ProductTypes    =    new    SelectList(await
_context.ProductTypes.ToListAsync(), "ProductTypeId", "Name");
                    ViewBag.MaterialTypes    =    new    SelectList(await
_context.MaterialTypes.ToListAsync(), "MaterialTypeId", "Name");

                    return View(products);
                }

                // ==================== МОДУЛЬ 3: Получение данных для
редактирования (для модалки) ====================
                [HttpGet]
                public async Task<IActionResult> GetProductForEdit(int
id)
                {
                    var product = await _context.Products
                        .Include(p => p.ProductType)
                        .Include(p => p.MainMaterial)
                        .FirstOrDefaultAsync(p => p.ProductId == id);

                    if (product == null)
                    {
                        return NotFound();
                    }

                    var viewModel = new ProductFormViewModel
                    {
                        ProductId = product.ProductId,
                        Article = product.Article,
                        ProductTypeId = product.ProductTypeId,
                        Name = product.Name,
                        MinPriceForPartner                              =
product.MinPriceForPartner,
                        MainMaterialId = product.MainMaterialId
                    };

                    return Json(viewModel);
                }

                //    ====================    МОДУЛЬ    3:    Сохранение
(добавление/редактирование) ====================
                [HttpPost]
                [ValidateAntiForgeryToken]
```

```
            public async Task<IActionResult> SaveProduct([FromForm]
ProductFormViewModel model)
            {
                // Проверка валидации модели
                if (!ModelState.IsValid)
                {
                    var errors = ModelState.Values.SelectMany(v =>
v.Errors).Select(e => e.ErrorMessage).ToList();
                    TempData["ErrorMessage"] = "Ошибка валидации: "
+ string.Join("; ", errors);
                    return RedirectToAction(nameof(Index));
                }

                // Проверка существования типов
                var productTypeExists = await
_context.ProductTypes.AnyAsync(pt => pt.ProductTypeId ==
model.ProductTypeId);
                var materialTypeExists = await
_context.MaterialTypes.AnyAsync(mt => mt.MaterialTypeId ==
model.MainMaterialId);

                if (!productTypeExists || !materialTypeExists)
                {
                    TempData["ErrorMessage"] = "Выбранный тип
продукции или материал не существует.";
                    return RedirectToAction(nameof(Index));
                }

                try
                {
                    if (model.ProductId == 0) // Добавление
                    {
                        var product = new Product
                        {
                            Article = model.Article,
                            ProductTypeId = model.ProductTypeId,
                            Name = model.Name,
                            MinPriceForPartner =
model.MinPriceForPartner,
                            MainMaterialId = model.MainMaterialId
                        };

                        _context.Products.Add(product);
                        await _context.SaveChangesAsync();
                        TempData["SuccessMessage"] = "Продукт
успешно добавлен.";
                    }
                    else // Редактирование
                    {
                        var product = await
_context.Products.FindAsync(model.ProductId);
                        if (product == null)
                        {
```

```
                                TempData["ErrorMessage"] = "Продукт не
найден.";
                                return RedirectToAction(nameof(Index));
                            }

                            product.Article = model.Article;
                            product.ProductTypeId                    =
model.ProductTypeId;
                            product.Name = model.Name;
                            product.MinPriceForPartner               =
model.MinPriceForPartner;
                            product.MainMaterialId                   =
model.MainMaterialId;

                            _context.Products.Update(product);
                            await _context.SaveChangesAsync();
                            TempData["SuccessMessage]    =    "Продукт
успешно обновлён.";
                        }
                    }
                    catch (Exception ex)
                    {
                        TempData["ErrorMessage"]    =    $"Ошибка    при
сохранении: {ex.Message}";
                    }

                    return RedirectToAction(nameof(Index));
                }

                // ==================== МОДУЛЬ 4: Страница цехов для
продукта ====================
                public    async    Task<IActionResult>    Workshops(int
productId)
                {
                    var product = await _context.Products
                        .Include(p => p.ProductWorkshops)
                            .ThenInclude(pw => pw.Workshop)
                        .FirstOrDefaultAsync(p    =>    p.ProductId    ==
productId);

                    if (product == null)
                    {
                        TempData["ErrorMessage"]    =    "Продукт    не
найден.";
                        return RedirectToAction(nameof(Index));
                    }

                    var viewModel = new ProductWorkshopsViewModel
                    {
                        ProductName = product.Name,
                        Workshops  =  product.ProductWorkshops.Select(pw
=> new WorkshopDetailViewModel
                        {
```

```
                        WorkshopName = pw.Workshop.Name,
                        EmployeeCount = pw.Workshop.EmployeeCount,
                        TimeHours = pw.TimeHours
                    }).ToList()
                };

                return View(viewModel);
            }

            // ==================== МОДУЛЬ 4: Страница калькулятора
сырья (GET - отобразить форму) ====================
            [HttpGet]
            public async Task<IActionResult> Calculate()
            {
                // Загружаем списки для выпадающих списков
                ViewBag.ProductTypes    =    new    SelectList(await
_context.ProductTypes.ToListAsync(), "ProductTypeId", "Name");
                ViewBag.MaterialTypes    =    new    SelectList(await
_context.MaterialTypes.ToListAsync(), "MaterialTypeId", "Name");

                return View(new RawMaterialCalculateViewModel());
            }

            // ==================== МОДУЛЬ 4: Расчёт сырья (POST -
обработка формы) ====================
            [HttpPost]
            [ValidateAntiForgeryToken]
            public            async            Task<IActionResult>
Calculate(RawMaterialCalculateViewModel model)
            {
                // Загружаем списки снова на случай ошибки
                ViewBag.ProductTypes    =    new    SelectList(await
_context.ProductTypes.ToListAsync(),     "ProductTypeId",     "Name",
model.ProductTypeId);
                ViewBag.MaterialTypes    =    new    SelectList(await
_context.MaterialTypes.ToListAsync(),    "MaterialTypeId",    "Name",
model.MaterialTypeId);

                if (!ModelState.IsValid)
                {
                    return View(model);
                }

                // Вызов сервиса расчёта
                int result = _calculator.CalculateRawMaterial(
                    model.ProductTypeId,
                    model.MaterialTypeId,
                    model.Quantity,
                    model.Param1,
                    model.Param2
                );

                if (result == -1)
```

```
                    {
                        ModelState.AddModelError("",     "Не     удалось
выполнить расчёт. Проверьте правильность введённых данных.");
                        return View(model);
                    }

                    model.Result = result;
                    return View(model);
                }
            }
        }
```